

Name: Aditty Kumar Chowdhury and

MD.MONIRUZZAMAN

ID: 22301518 and 22101197

Section: 06

Group: 10

Course: CSE422 (Artificial Intelligence)

Project title: Stroke risk prediction and patient
risk group analysis using machine learning

Table of Contents

Introduction	1
Dataset Description.....	2
Dataset pre-processing.....	13
Dataset splitting.....	19
Model training and testing (Supervised).....	21
Model training and testing (Unsupervised).....	29
Model selection/ Comparison analysis	33
Conclusion	43

Introduction:

Stroke is a serious health condition in today's day and age. It can occur when the blood supply to a part of the brain is interrupted and so early identification of stroke risk can help improve awareness and also support preventive healthcare [decisions](#). In this project our goal is to build a machine learning system that predicts whether a person is likely to have a stroke by using data related to health, demographic and lifestyle related information.

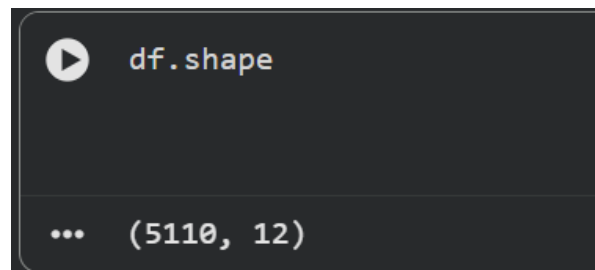
The dataset we have contains patient information such as age, gender, hypertension, heart disease, ever_married, work_type, Residence_type, average glucose level, BMI and smoking status. The target variable in our dataset is stroke which indicates whether the patient had a stroke or not.

The main objective of this project is to compare different machine learning models for stroke prediction. The output only has two possible classes which are 1 (yes) or 0 (No) so this is a binary classification problem. We will train and evaluate several supervised learning models such as logistic regression, KNN, Naive Bayes and Neural Network. In addition, K-Means clustering will be used to analyse patient risk groups for unsupervised learning.

The motivation behind our project is to explore how artificial intelligence can be used to detect meaningful patterns in healthcare data.

Dataset description:

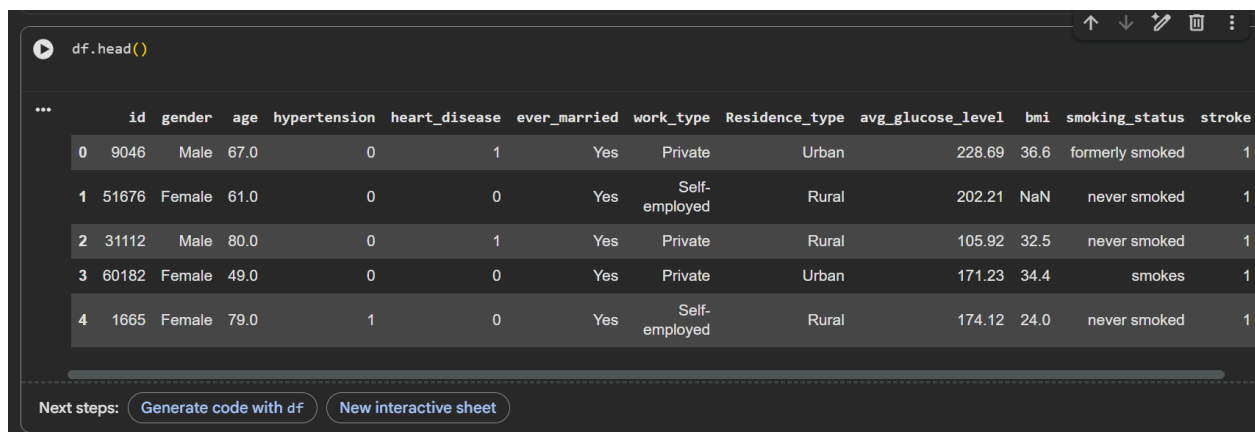
The main goal of using this dataset is to predict whether a patient has experienced a stroke or not based on these available features. There are 5110 datapoints in our dataset and 12 columns in total.



```
df.shape
```

```
(5110, 12)
```

Figure 1: Datapoints and columns



```
df.head()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Next steps: [Generate code with df](#) [New interactive sheet](#)

Figure 2: First five rows of the dataset

Among these columns our target or rather output will be stroke. The remaining columns describe different characteristics of the patients. The id column is only a unique identifier for each patient so it does not provide useful information for prediction and will not be used as an input feature during model training.

After excluding the “id” column and the target column “stroke” our dataset contains 10 input features. The features are gender, age, hypertension, heart_disease, ever married, work_type, Residence_type, avg_glucose_level, bmi and smoking_status. The output feature is stroke which

contains two possible values which are “0” which means the patient did not have a stroke and “1” which means the patient had a stroke.

This project is a classification problem because the target variable is categorical. The model does not predict a continuous numerical value. Instead it predicts one of the two outcomes which are stroke or no stroke. Since there are only two possible outputs this is a binary classification problem.

The dataset contains both quantitative and categorical features. The quantitative features are age, avg_glucose_level and bmi because these features contain continuous numerical values. The features hypertension and heart_disease are binary numerical features because they are already represented using 0 and 1. The categorical features are gender, ever_married, work_type, Residence_type and smoking_status because they contain text based categories. For example Residence_type feature contains categories such as Urban and Rural. Smoking_status feature contains categories such as formerly smoked, never smoked and unknown.

```
df.info
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     5110 non-null   int64
1   gender                 5110 non-null   object
2   age                    5110 non-null   float64
3   hypertension           5110 non-null   int64
4   heart_disease          5110 non-null   int64
5   ever_married           5110 non-null   object
6   work_type              5110 non-null   object
7   Residence_type         5110 non-null   object
8   avg_glucose_level      5110 non-null   float64
9   bmi                    4909 non-null   float64
10  smoking_status         5110 non-null   object
11  stroke                  5110 non-null   int64
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

Figure 3: Dataset information showing column names, data types and non-null values

The categorical variables need to be encoded before model training because machine learning algorithms do not understand text values and we will need to use algorithms such as logistic

regression, KNN, naive bayes and neural networks which will need numerical input. Therefore categorical features such as gender, ever_married, work_type, Residence_type and smoking_status will be converted into numerical form using encoding techniques.

We also checked for missing values in our dataset. From our analysis only BMI column contains missing values. There are 201 missing values in BMI column but in the other features there are no null values. Since BMI is a numerical feature this missing values can be handled in the data preprocessing section using an appropriate imputation technique.

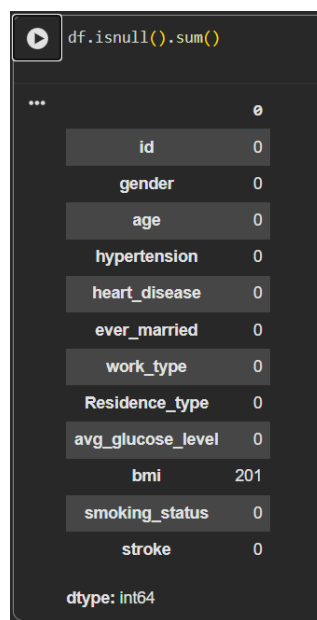


Figure 4: Missing values in the dataset

A correlation heatmap will be used to observe the relationship between the input features of our dataset and the output feature stroke. From the correlation analysis we will be able to understand which features have stronger or weaker relationships with stroke occurrence. In our dataset features such as age, heart_disease, hypertension and avg_glucose_level are expected to show stronger positive relationship with stroke compared to other features. This means that older patients, patients with heart disease, patients with hypertension and patients with a higher glucose may have a tendency to be associated with stroke cases in this dataset. However,

correlation does not prove causation. It only shows the strength and direction of a relationship between the variables.

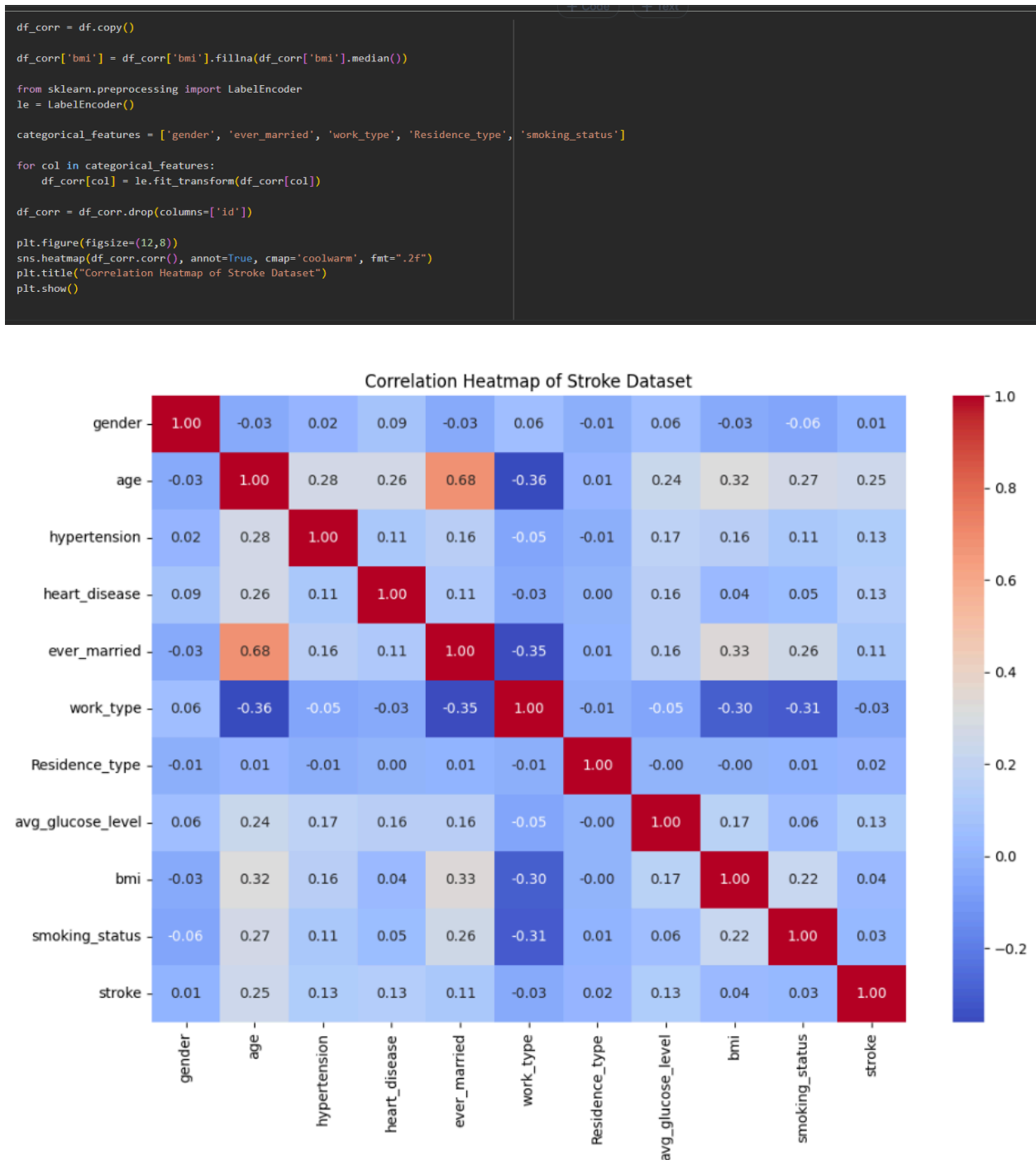


Figure 5: Correlation heatmap of input and output features

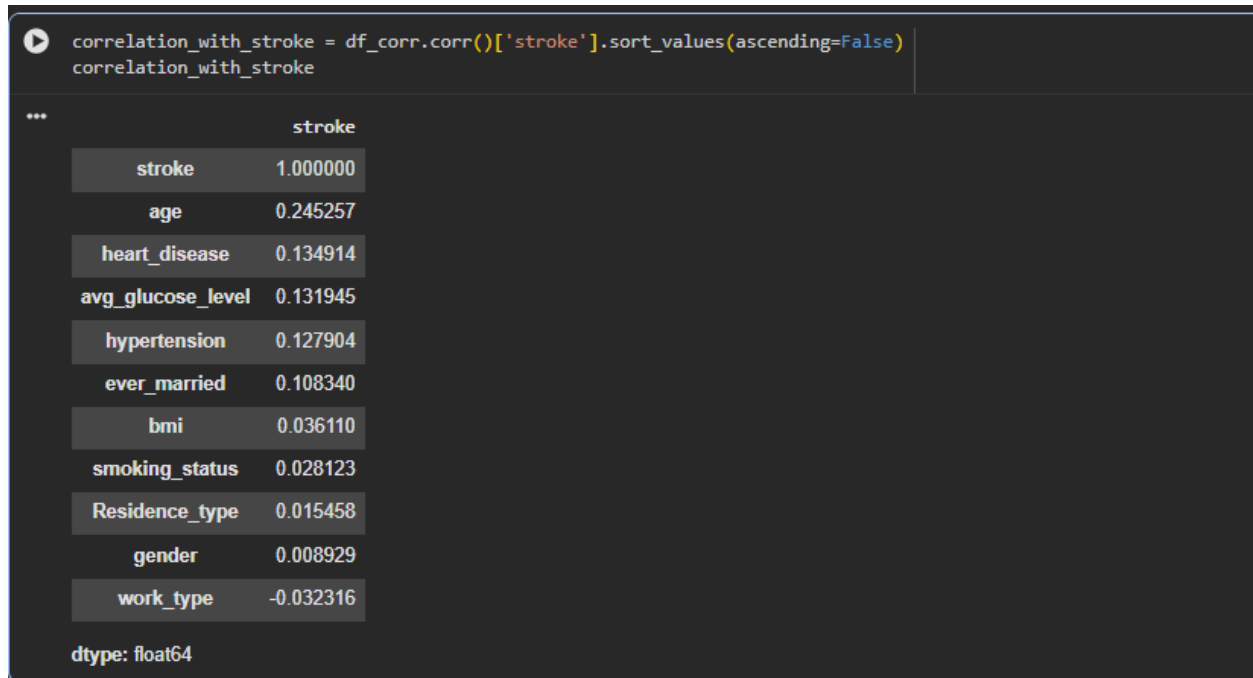


Figure 6: Correlation values of all features with the target variable stroke

From the correlation heatmap age shows the strongest positive correlation with stroke with a correlation value of 0.245257. This suggests that stroke is more strongly correlated with age compared to other features in our dataset. The features heart_disease, avg_glucose_level and hypertension also show positive correlation with target stroke with values of 0.134914, 0.131945 and 0.127904 respectively. This means that patients with heart disease, higher average glucose and hypertension may have a higher tendency to be associated with stroke cases. However these correlation values are not very high so the relationships are weak to moderate. Other features show very low correlation with stroke and so the correlation test suggests that age, heart_disease, avg_glucose_level and hypertension may be more useful features for stroke prediction but correlation only shows relationship and does not prove direct cause.

Our dataset is highly imbalanced. The output feature stroke has 4861 patients with no stroke and only 249 patients with stroke. So clearly the output feature stroke does not have an equal number of instances for each class. So 95.13% of our data belongs to the no stroke class while only

4.87% belongs to the stroke class. A barchart will be used to represent the number of instances in each class where 0 class will stand for no stroke and 1 for stroke.

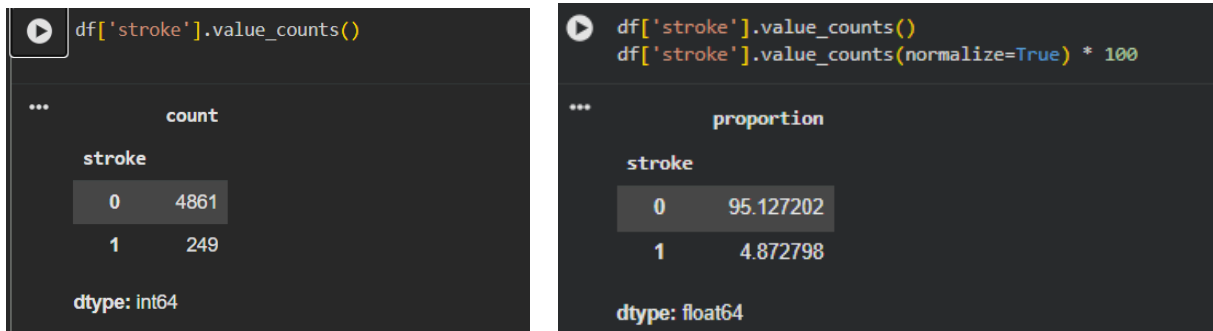


Figure 7: Class distribution of the target variable stroke



Figure 8: Bar chart showing class distribution of the target variable stroke

The class imbalance is important because it can affect our model's performance. Our model might achieve a high accuracy by predicting most patients as no stroke but that does not mean the model is good at identifying actual stroke cases. For this reason, accuracy alone will not be enough for us to evaluate the models and so we will use precision, recall, f1-score, confusion matrix, ROC curve and AUC score for model evaluation.

EDA or exploratory data analysis will be performed to identify important patterns and relationships in the dataset. We will analyze how stroke is related to features such as age, average glucose level, BMI, hypertension, heart disease, smoking status and work type. For example age distribution can show whether stroke cases are more common among older patients. The relationship between hypertension and stroke can show whether patients with hypertension have a higher stroke occurrence. Similarly average glucose level and BMI can help us understand whether these health related numerical features show noticeable differences between stroke and no stroke patients. These results from EDA will help us to understand our dataset better before applying machine learning models.

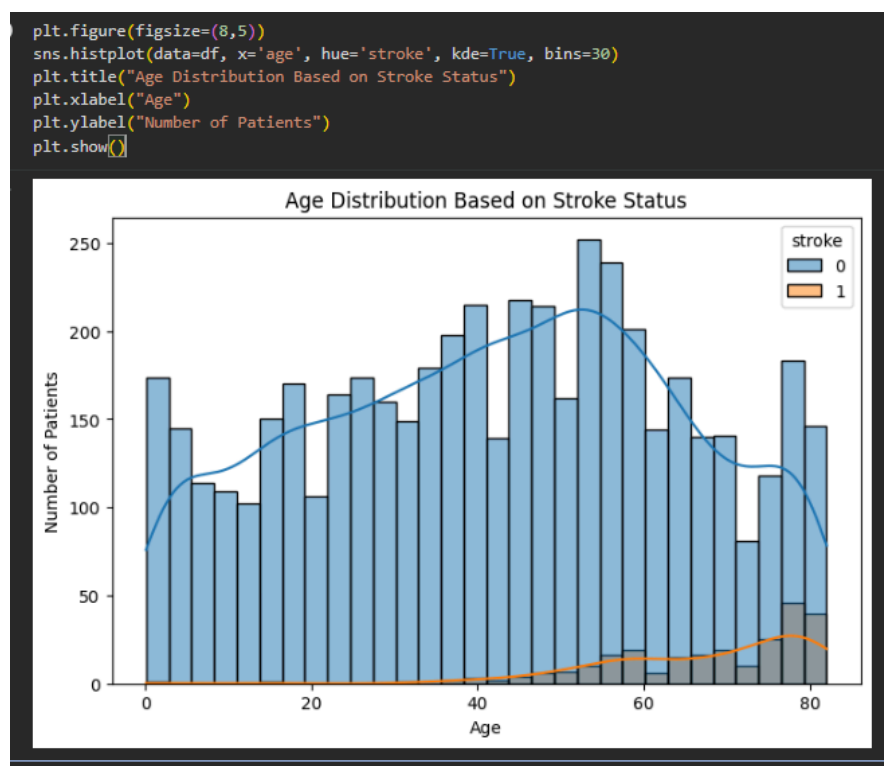


Figure 9: Age distribution based on stroke status

From the age distribution we can see that stroke cases appear more frequently among older patients compared to younger patients. This supports the correlation result where age had the strongest positive relationship with the target variable stroke.

```
plt.figure(figsize=(8,5))
sns.boxplot(x='stroke', y='avg_glucose_level', data=df)
plt.title("Average Glucose Level Based on Stroke Status")
plt.xlabel("Stroke Class")
plt.ylabel("Average Glucose Level")
plt.show()
```

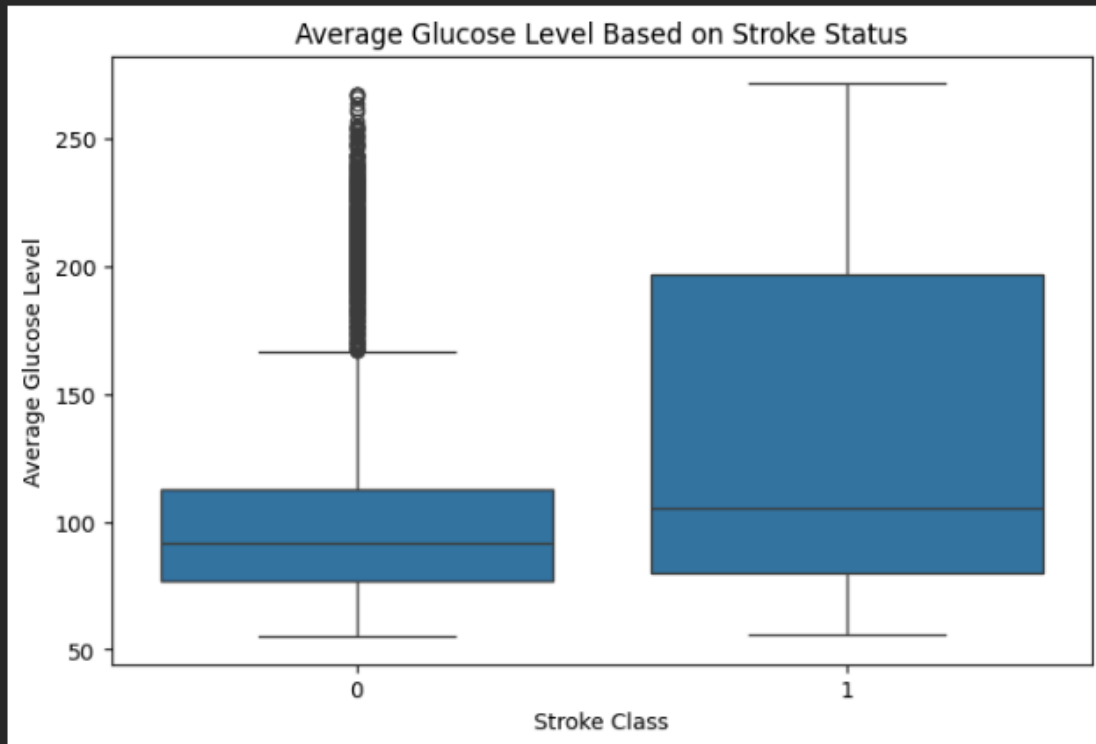


Figure 10: Average glucose level based on stroke status

The boxplot shows that patients who had a stroke generally have a higher average glucose level compared to patients who did not have a stroke. This suggests that average glucose level may be an important health feature for stroke prediction.

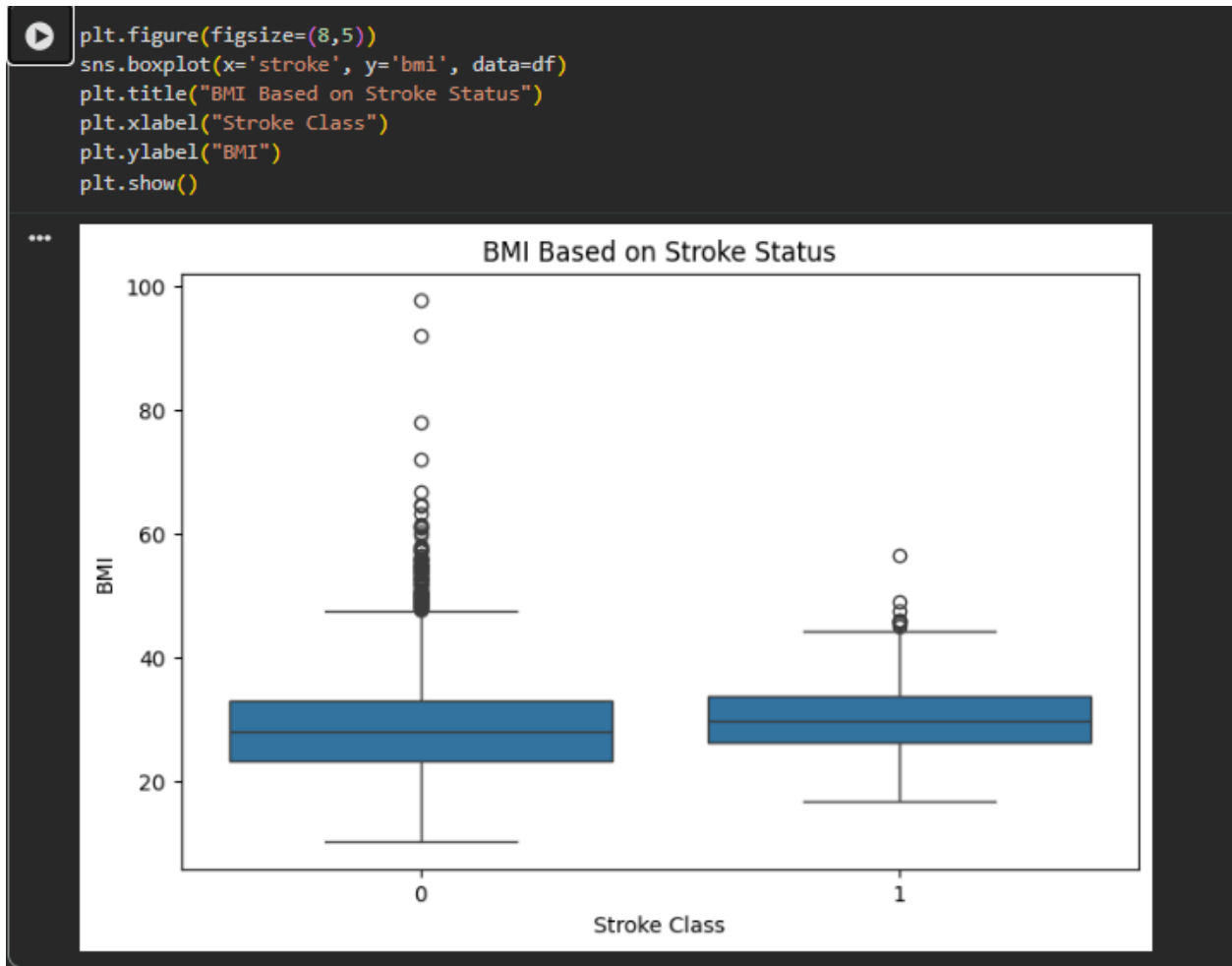


Figure 11: BMI based on stroke status

The BMI distribution shows some differences between stroke and no stroke patients but the relationship is not as strong as age or average glucose level. This also matches the correlation result where BMI had a very low correlation with the target variable.

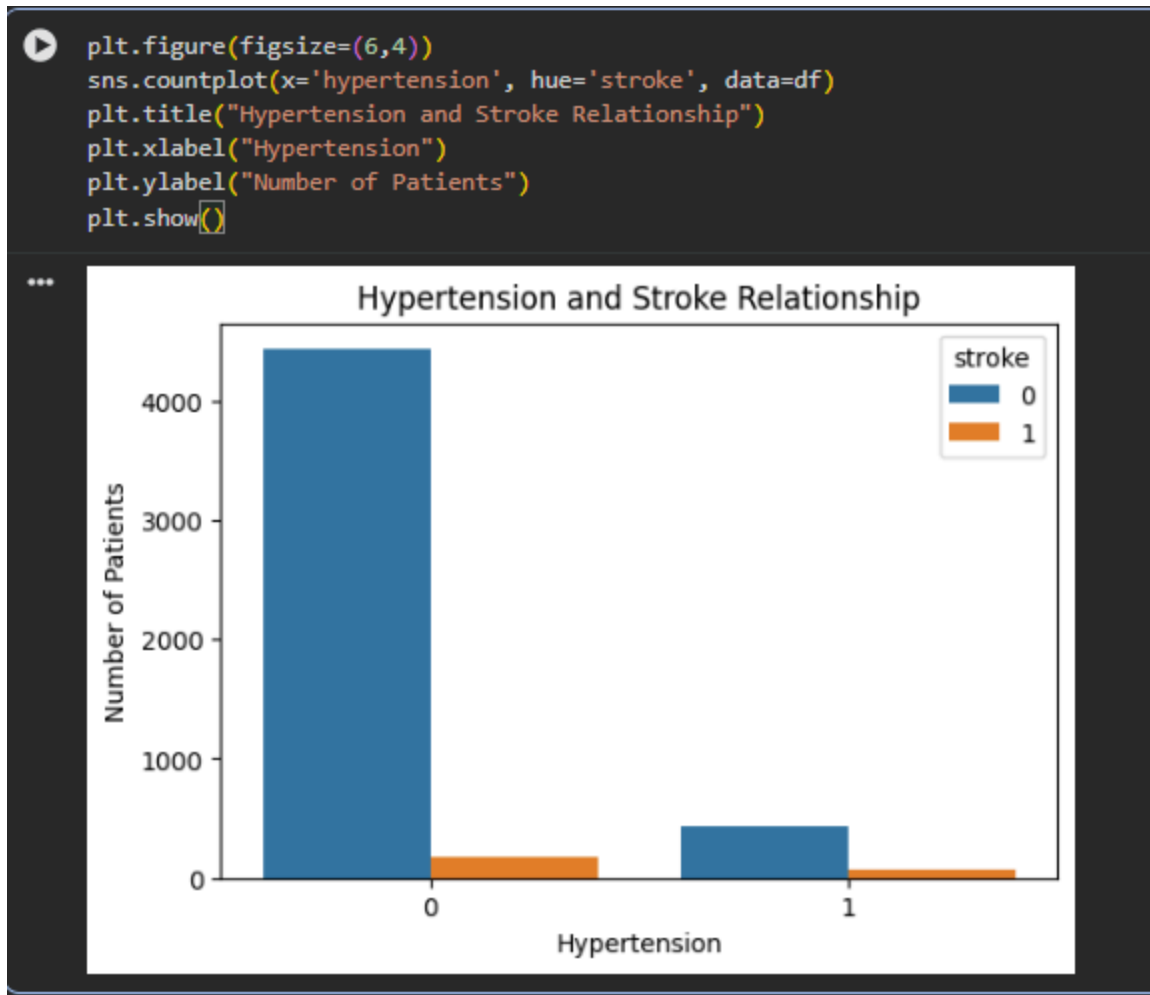


Figure 12: Relationship between hypertension and stroke

The graph shows stroke cases in both groups but hypertension has a positive relationship with stroke in the dataset. This supports the correlation heatmap where hypertension shows a positive correlation with the target variable.

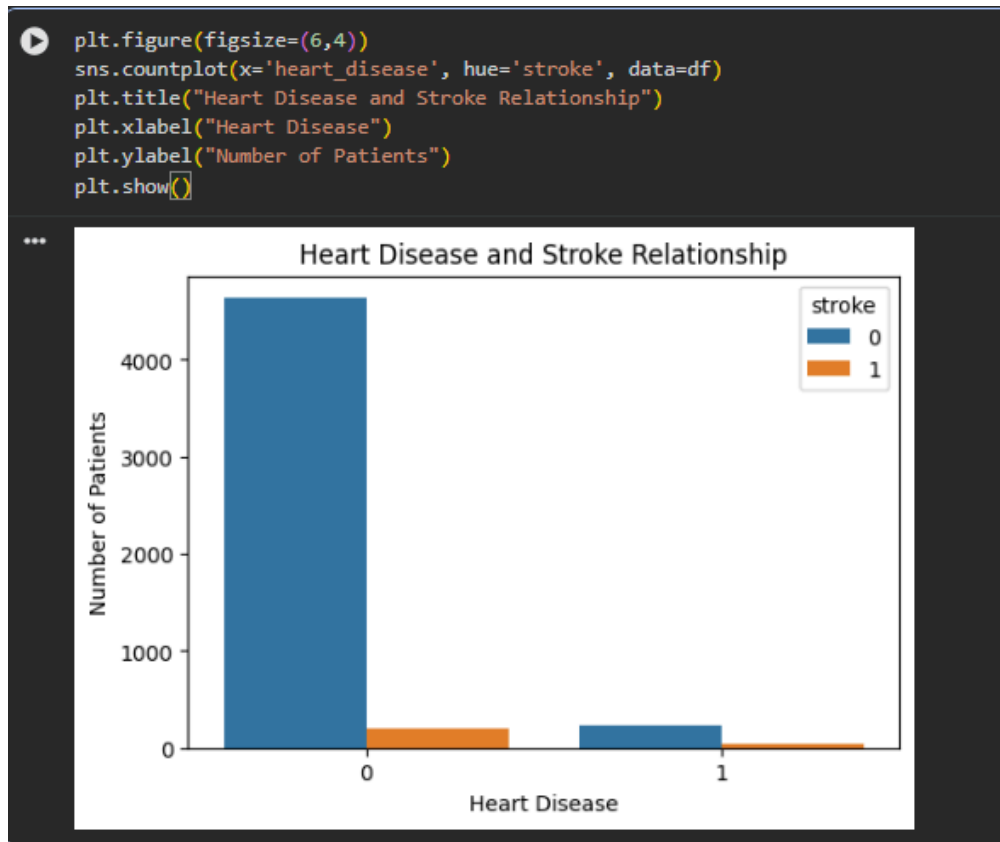


Figure 13: Relationship between heart disease and stroke

The graph shows that heart disease is related to stroke occurrence in the dataset. Although the number of patients with heart disease is smaller the feature still shows a positive relationship with stroke in the correlation analysis.

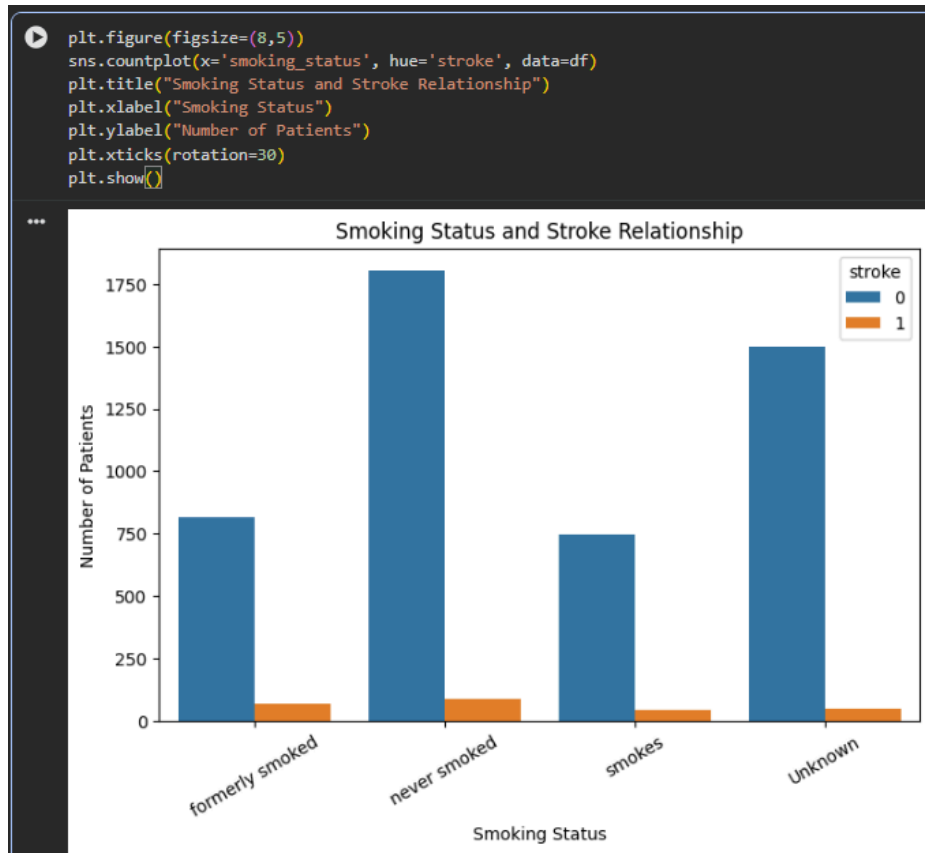


Figure 14: Relationship between smoking status and stroke

The graph shows how stroke cases are distributed among different smoking status categories. Although stroke cases are present in multiple smoking groups but the relationship are not as strong as age, heart disease, hypertension or average glucose level. This also matches the correlation result where `smoking_status` had a very low correlation with stroke.

Dataset pre-processing:

Before applying preprocessing techniques we made a copy of the original dataset that was created. This helps us keep the original dataset unchanged while performing preprocessing on a separate dataframe.

```
[46] ✓ 0s df_processed[['id']].head()
```

	id
0	9046
1	51676
2	31112
3	60182
4	1665

Figure 15: The id column before preprocessing

```
df_processed = df_processed.drop(columns=['id'])
df_processed.head()
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Figure 16: Dataset after removing the id column

The first preprocessing fault is the id column. The id column only works as a unique identifier for each patient but it does not provide us with any meaningful health, demographic or lifestyle information for stroke prediction. If this column is kept during model training it may add unnecessary noise to the dataset. Hence, as a solution, we decided to remove the id column from the dataset.


```
df_processed.isnull().sum()
```

gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	201
smoking_status	0
stroke	0

dtype: int64

Figure 17: Missing values before imputation

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

imputer = SimpleImputer(strategy='median')

df_processed[['bmi']] = imputer.fit_transform(df_processed[['bmi']])

print("Median value used for BMI:", imputer.statistics_[0])

df_processed.isnull().sum()
```

Median value used for BMI: 28.1

0	
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	0
smoking_status	0
stroke	0

dtype: int64

Figure 18: Missing values after median imputation using SimpleImputer

The second fault we found was missing values in bmi feature column. The bmi column contains 201 missing values but the other features do not have any missing values. Since this is not the majority of dataset dropping the whole bmi column would remove useful information. Dropping

rows with missing bmi values could reduce the number of data points. As a solution, we used median imputation using SimpleImputer from sklearn. The median strategy was selected because bmi is a numerical feature and median is less affected by outliers compared to mean. After applying median imputation our dataset no longer has missing values in the bmi column.

```
categorical_columns = df_processed.select_dtypes(include=['object']).columns
print("Categorical columns:")
print(categorical_columns)

... Categorical columns:
Index(['gender', 'ever_married', 'work_type', 'Residence_type',
      'smoking_status'],
      dtype='object')

for col in categorical_columns:
    print(col)
    print(df_processed[col].unique())
    print()

gender
['Male' 'Female' 'Other']

ever_married
['Yes' 'No']

work_type
['Private' 'Self-employed' 'Govt_job' 'children' 'Never_worked']

Residence_type
['Urban' 'Rural']

smoking_status
['formerly smoked' 'never smoked' 'smokes' 'Unknown']
```

Figure 19: Categorical columns in the dataset and Unique values of categorical columns

```
try:
    encoder = OneHotEncoder(drop='first', sparse_output=False)
except TypeError:
    encoder = OneHotEncoder(drop='first', sparse=True)

encoded_data = encoder.fit_transform(df_processed[categorical_columns])
encoded_columns = encoder.get_feature_names_out(categorical_columns)

encoded_df = pd.DataFrame(
    encoded_data,
    columns=encoded_columns,
    index=df_processed.index)

df_processed = df_processed.drop(columns=categorical_columns)
df_processed = pd.concat([df_processed, encoded_df], axis=1)
df_processed.head()
```

```
...
   age  hypertension  heart_disease  avg_glucose_level  bmi  stroke  gender_Male  gender_Other  ever_married_yes  work_type_Never_worked  work_type_Private  work_type_Self-employed  work_type_children  Residence_type_Urban  smoking_status_formerly_smoked  smoking_status_never_smoked  smoking_status_smokes
0  67.0             0             1         228.69  36.6      1         1.0           0.0             1.0             0.0             1.0             0.0             0.0             1.0             1.0             0.0             0.0
1  61.0             0             0         202.21  28.1      1         0.0           0.0             1.0             0.0             0.0             1.0             0.0             0.0             0.0             1.0             0.0
2  80.0             0             1         195.92  32.5      1         1.0           0.0             1.0             0.0             1.0             0.0             0.0             0.0             0.0             1.0             0.0
3  49.0             0             0         171.23  34.4      1         0.0           0.0             1.0             0.0             1.0             0.0             0.0             1.0             0.0             0.0             1.0
4  79.0             1             0         174.12  24.8      1         0.0           0.0             1.0             0.0             0.0             1.0             0.0             0.0             0.0             1.0             0.0
```

Figure 20: Dataset after applying One-Hot Encoding

```
df_processed.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                  5110 non-null   float64
1   hypertension                        5110 non-null   int64
2   heart_disease                      5110 non-null   int64
3   avg_glucose_level                  5110 non-null   float64
4   bmi                                5110 non-null   float64
5   stroke                             5110 non-null   int64
6   gender_Male                        5110 non-null   float64
7   gender_Other                       5110 non-null   float64
8   ever_married_Yes                   5110 non-null   float64
9   work_type_Never_worked              5110 non-null   float64
10  work_type_Private                   5110 non-null   float64
11  work_type_Self-employed              5110 non-null   float64
12  work_type_children                  5110 non-null   float64
13  Residence_type_Urban                5110 non-null   float64
14  smoking_status_formerly smoked      5110 non-null   float64
15  smoking_status_never smoked         5110 non-null   float64
16  smoking_status_smokes               5110 non-null   float64
dtypes: float64(14), int64(3)
memory usage: 678.8 KB
```

Figure 21: Dataset information after encoding categorical features

The third preprocessing fault we found was the presence of categorical values. Columns such as gender, ever_married, work_type, Residence_type and smoking status contain text based categories. Machine learning models require numerical input so these categorical values cannot be used directly during model training. To solve this we used one hot encoding using sklearn's OneHotEncoder. We chose one hot encoding because these categorical features do not have a natural order or ranking. For example one of the features is Residence type and it contains Urban and Rural. Another feature named smoking_status contains categories such as smoked, never smoked, smokes and unknown. One hot encoding avoids this issue by creating separate numerical columns for categories.

```
df_processed[['age', 'avg_glucose_level', 'bmi']].describe()
```

	age	avg_glucose_level	bmi
count	5110.000000	5110.000000	5110.000000
mean	43.226614	106.147677	28.862035
std	22.612647	45.283560	7.699562
min	0.080000	55.120000	10.300000
25%	25.000000	77.245000	23.800000
50%	45.000000	91.885000	28.100000
75%	61.000000	114.090000	32.800000
max	82.000000	271.740000	97.600000

Figure 22: Different ranges of numerical features before scaling

```
features_to_scale = ['age', 'avg_glucose_level', 'bmi']
scaler = StandardScaler()
print("features selected for scaling:", features_to_scale)
# Features selected for scaling: ['age', 'avg_glucose_level', 'bmi']

df_processed.head()
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke	gender_male	gender_other	ever_married_yes	work_type_never_worked	work_type_private	work_type_self-employed	work_type_children	Residence_type_urban	smoking_status_formerly_smoked	smoking_status_never_smoked	smoking_status_smokes
0	67.0	0	1	228.69	36.6	1	1.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0
1	61.0	0	0	202.21	26.1	1	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0
2	80.0	0	1	105.92	32.5	1	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
3	49.0	0	0	171.23	34.4	1	0.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
4	79.0	1	0	174.12	24.0	1	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0

Figure 23: Numerical features selected for StandardScaler

The fourth preprocessing fault we found was that some features have different ranges. For example age. Avg_glucose_level and bmi are measured on different scales. This can affect models such as KNN, Logistic Regression and Neural Network because these models are sensitive to feature magnitude. KNN uses distance based calculation so a feature with a larger numerical range can dominate the distance calculation. As a solution, we applied feature scaling using sklearn's StandardScaler. The actual scaling has been applied after dataset splitting so that the scaler was fitted only on the training data. StandardScaler was selected because it transforms numerical features so that they have a mean close to 0 and a standard deviation close to 1. This is suitable for models such as KNN, Logistic Regression and Neural Network. In our project the only continuous numerical features are age, avg_glucose_level and bmi so they were selected for

scaling. Binary and one hot encoding features were not selected for scaling because they already represent 0 and 1.

Dataset splitting:

After completing dataset preprocessing we divided our dataset into input features and output target. The input features set “x” contains all the selected and preprocessed features while the output variable “y” contains the target column stroke.

The dataset we have solves a classification problem and the target variable itself is highly imbalanced so we used stratified splitting instead of simple random splitting. The dataset contains many more no stroke cases than stroke cases. If normal random splitting was used the training or test set might not contain a proper proportion of stroke cases. That's why we used “stratify=y” to maintain a similar class distribution in both the training and testing sets.

```
print("Original target distribution:")
print(y.value_counts(normalize=True) * 100)

print("\nTraining target distribution:")
print(y_train.value_counts(normalize=True) * 100)

print("\nTesting target distribution:")
print(y_test.value_counts(normalize=True) * 100)
```

... Original target distribution:
stroke
0 95.127202
1 4.872798
Name: proportion, dtype: float64

Training target distribution:
stroke
0 95.132094
1 4.867906
Name: proportion, dtype: float64

Testing target distribution:
stroke
0 95.107632
1 4.892368
Name: proportion, dtype: float64

Figure 24: Target class distribution after stratified splitting

In our project we used 80 percent of our data for training and 20 percent of our data for testing.

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=1,
    stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

... X_train shape: (4088, 16)
X_test shape: (1022, 16)
y_train shape: (4088,)
y_test shape: (1022,)

```

Figure 25: Shape of training and testing datasets

We did not use a separate validation set because we do not require a hyperparameter tuning. For Neural Network model validation can be monitored later using a validation split from the training data during model training.

After splitting the dataset we applied feature scaling using StandardScaler. The scaler was fitted only on training data and then used to transform both the training data and the testing data.

```

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[features_to_scale] = scaler.fit_transform(X_train[features_to_scale])
X_test_scaled[features_to_scale] = scaler.transform(X_test[features_to_scale])

X_train_scaled.head()

```

	age	hypertension	heart_disease	avg_glucose_level	bmi	gender_male	gender_other	ever_married_Yes	work_type_Never_worked	work_type_Private	work_type_Self-employed	work_type_children	residence_type_urban	smoking_status_formerly_smoked	smoking_status_never_smoked	smoking_status_smokes
2051	-0.347972	0	0	-0.702666	-0.832056	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
2065	-1.052353	0	0	-0.729577	-0.756441	1.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0
1201	0.136290	0	0	-0.464433	0.435385	1.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
3918	1.016767	0	0	-0.006731	0.357230	1.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0
449	-0.039805	0	0	-0.843956	0.291767	1.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0

Figure 26: Training data after applying StandardScaler

This was done to avoid data leakage because the test data should not influence the scaling parameter that was learned from the training data.

```

X_train_scaled[features_to_scale].describe()

```

	age	avg_glucose_level	bmi
count	4.088000e+03	4.088000e+03	4.088000e+03
mean	4.171484e-17	8.169156e-17	5.301261e-17
std	1.000122e+00	1.000122e+00	1.000122e+00
min	-1.885284e+00	-1.131256e+00	-2.289444e+00
25%	-7.882100e-01	-6.420612e-01	-6.647225e-01
50%	4.824271e-02	-3.170891e-01	-8.820851e-02
75%	7.966478e-01	1.759110e-01	5.145106e-01
max	1.721148e+00	3.646985e+00	8.284347e+00

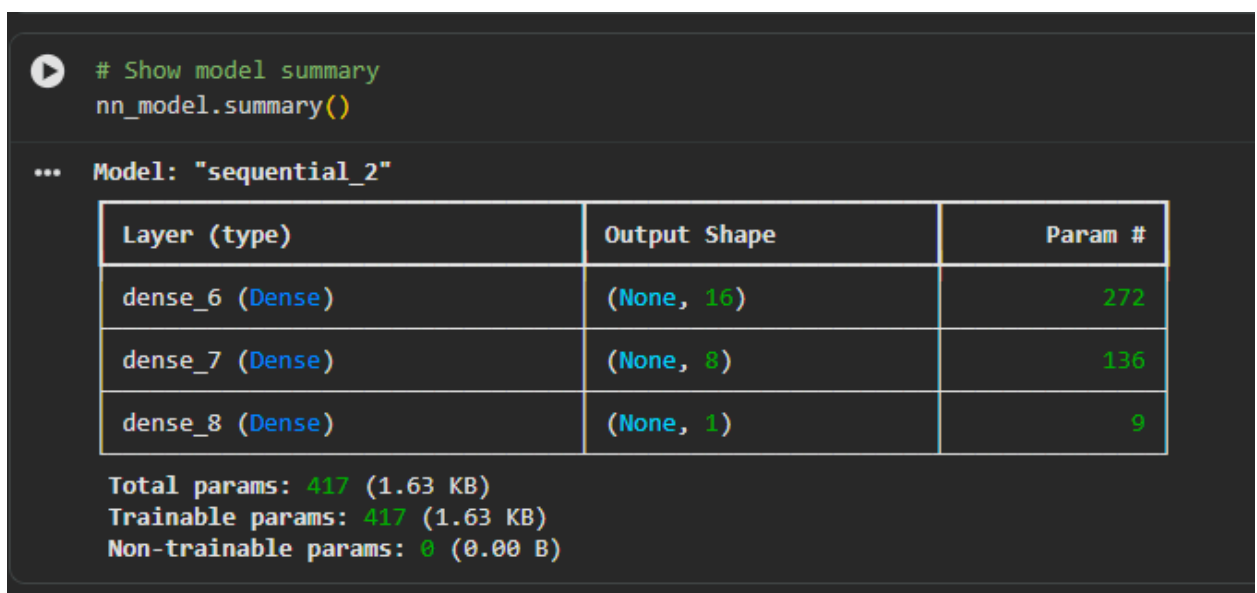
Figure 27: Scaled numerical feature statistics

Model training and testing (Supervised)

1) Neural network:

A neural network was also applied because the project guideline requires the use of a neural network model and since our project is a binary classification problem where the output is only 0 for no stroke and 1 for stroke we can use the output layer of Neural Network with sigmoid function.

Our input shape was based on the number of input features after preprocessing. The model used dense layers where the hidden layers used ReLU activation function and the output layer used sigmoid activation function.



```
# Show model summary
nn_model.summary()
```

... Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 16)	272
dense_7 (Dense)	(None, 8)	136
dense_8 (Dense)	(None, 1)	9

Total params: 417 (1.63 KB)
Trainable params: 417 (1.63 KB)
Non-trainable params: 0 (0.00 B)

Figure 28: Neural Network model summary

The neural network was compiled using Adam optimizer and binary cross entropy loss function. Binary cross entropy was used because our project is based on binary classification. During training, accuracy, precision and recall were used as metrics. At first our neural network gave high accuracy but precision, recall and f1-score were 0. This basically meant that the model predicted no stroke class since it was the majority. As

our dataset is highly imbalanced we used class weights during neural network training to give more importance to the minority stroke class.

```
# Build Neural Network model
# Compute class weights to handle imbalance
classes = np.unique(y_train)

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,
    y=y_train
)

class_weight_dict = dict(zip(classes, class_weights))

print("Class weights:", class_weight_dict)

... Class weights: {np.int64(0): np.float64(0.5255849832861919), np.int64(1): np.float64(10.271356783919598)}
```

Figure 29: Class weights used for Neural Network training

During Neural network training we used class weights to handle the imbalance in target variable. Since the stroke class has far fewer samples than the no stroke class, class weighting gave more importance to the minority stroke class during training. The training output shows that recall improved while the model was learning. This means the neural network no longer only focused on the majority no stroke class.


```

Epoch 1/30
103/103 ----- 3s 9ms/step - accuracy: 0.4043 - loss: 0.6633 - precision: 0.0611 - recall: 0.8224 - val_accuracy: 0.7188 - val_loss: 0.6225 - val_precision: 0.1355 - val_recall: 0.7234
Epoch 2/30
103/103 ----- 1s 11ms/step - accuracy: 0.6988 - loss: 0.5790 - precision: 0.1111 - recall: 0.7829 - val_accuracy: 0.7396 - val_loss: 0.5559 - val_precision: 0.1542 - val_recall: 0.7872
Epoch 3/30
103/103 ----- 1s 12ms/step - accuracy: 0.7278 - loss: 0.5294 - precision: 0.1211 - recall: 0.7763 - val_accuracy: 0.6822 - val_loss: 0.5754 - val_precision: 0.1340 - val_recall: 0.8298
Epoch 4/30
103/103 ----- 1s 9ms/step - accuracy: 0.7223 - loss: 0.4954 - precision: 0.1228 - recall: 0.8092 - val_accuracy: 0.7176 - val_loss: 0.5185 - val_precision: 0.1489 - val_recall: 0.8298
Epoch 5/30
103/103 ----- 1s 8ms/step - accuracy: 0.7251 - loss: 0.4761 - precision: 0.1276 - recall: 0.8421 - val_accuracy: 0.6968 - val_loss: 0.5369 - val_precision: 0.1398 - val_recall: 0.8298
Epoch 6/30
103/103 ----- 1s 6ms/step - accuracy: 0.7306 - loss: 0.4630 - precision: 0.1299 - recall: 0.8421 - val_accuracy: 0.7359 - val_loss: 0.4728 - val_precision: 0.1551 - val_recall: 0.8085
Epoch 7/30
103/103 ----- 1s 6ms/step - accuracy: 0.7367 - loss: 0.4561 - precision: 0.1311 - recall: 0.8289 - val_accuracy: 0.7200 - val_loss: 0.4821 - val_precision: 0.1473 - val_recall: 0.8085
Epoch 8/30
103/103 ----- 1s 8ms/step - accuracy: 0.7379 - loss: 0.4514 - precision: 0.1317 - recall: 0.8289 - val_accuracy: 0.7176 - val_loss: 0.4809 - val_precision: 0.1462 - val_recall: 0.8085
Epoch 9/30
103/103 ----- 1s 4ms/step - accuracy: 0.7352 - loss: 0.4470 - precision: 0.1335 - recall: 0.8553 - val_accuracy: 0.7445 - val_loss: 0.4484 - val_precision: 0.1597 - val_recall: 0.8085
Epoch 10/30
103/103 ----- 0s 3ms/step - accuracy: 0.7453 - loss: 0.4434 - precision: 0.1343 - recall: 0.8224 - val_accuracy: 0.7237 - val_loss: 0.4686 - val_precision: 0.1490 - val_recall: 0.8085
Epoch 11/30
103/103 ----- 0s 3ms/step - accuracy: 0.7401 - loss: 0.4401 - precision: 0.1334 - recall: 0.8355 - val_accuracy: 0.7469 - val_loss: 0.4379 - val_precision: 0.1618 - val_recall: 0.8085
Epoch 12/30
103/103 ----- 0s 3ms/step - accuracy: 0.7419 - loss: 0.4371 - precision: 0.1335 - recall: 0.8289 - val_accuracy: 0.7531 - val_loss: 0.4252 - val_precision: 0.1616 - val_recall: 0.7872
Epoch 13/30
103/103 ----- 0s 3ms/step - accuracy: 0.7483 - loss: 0.4357 - precision: 0.1373 - recall: 0.8355 - val_accuracy: 0.7262 - val_loss: 0.4583 - val_precision: 0.1502 - val_recall: 0.8085
Epoch 14/30
103/103 ----- 0s 3ms/step - accuracy: 0.7388 - loss: 0.4326 - precision: 0.1344 - recall: 0.8487 - val_accuracy: 0.7579 - val_loss: 0.4196 - val_precision: 0.1644 - val_recall: 0.7872
Epoch 15/30
103/103 ----- 0s 3ms/step - accuracy: 0.7465 - loss: 0.4312 - precision: 0.1364 - recall: 0.8355 - val_accuracy: 0.7628 - val_loss: 0.4121 - val_precision: 0.1674 - val_recall: 0.7872
Epoch 16/30
103/103 ----- 0s 3ms/step - accuracy: 0.7437 - loss: 0.4275 - precision: 0.1367 - recall: 0.8487 - val_accuracy: 0.7408 - val_loss: 0.4476 - val_precision: 0.1605 - val_recall: 0.8298
Epoch 17/30
103/103 ----- 0s 4ms/step - accuracy: 0.7511 - loss: 0.4261 - precision: 0.1402 - recall: 0.8487 - val_accuracy: 0.7323 - val_loss: 0.4608 - val_precision: 0.1587 - val_recall: 0.8511
Epoch 18/30
103/103 ----- 0s 4ms/step - accuracy: 0.7511 - loss: 0.4261 - precision: 0.1402 - recall: 0.8487 - val_accuracy: 0.7323 - val_loss: 0.4608 - val_precision: 0.1587 - val_recall: 0.8511
Epoch 19/30
103/103 ----- 0s 3ms/step - accuracy: 0.7505 - loss: 0.4223 - precision: 0.1407 - recall: 0.8553 - val_accuracy: 0.7200 - val_loss: 0.4842 - val_precision: 0.1527 - val_recall: 0.8511
Epoch 20/30
103/103 ----- 0s 3ms/step - accuracy: 0.7419 - loss: 0.4224 - precision: 0.1381 - recall: 0.8684 - val_accuracy: 0.7213 - val_loss: 0.4058 - val_precision: 0.1533 - val_recall: 0.8511
Epoch 21/30
103/103 ----- 0s 3ms/step - accuracy: 0.7410 - loss: 0.4191 - precision: 0.1391 - recall: 0.8816 - val_accuracy: 0.7359 - val_loss: 0.4535 - val_precision: 0.1579 - val_recall: 0.8298
Epoch 22/30
103/103 ----- 0s 3ms/step - accuracy: 0.7453 - loss: 0.4200 - precision: 0.1389 - recall: 0.8618 - val_accuracy: 0.7396 - val_loss: 0.4464 - val_precision: 0.1598 - val_recall: 0.8298
Epoch 23/30
103/103 ----- 0s 3ms/step - accuracy: 0.7486 - loss: 0.4163 - precision: 0.1398 - recall: 0.8553 - val_accuracy: 0.7396 - val_loss: 0.4387 - val_precision: 0.1598 - val_recall: 0.8298
Epoch 24/30
103/103 ----- 0s 4ms/step - accuracy: 0.7437 - loss: 0.4128 - precision: 0.1397 - recall: 0.8750 - val_accuracy: 0.7384 - val_loss: 0.4449 - val_precision: 0.1564 - val_recall: 0.8085
Epoch 25/30
103/103 ----- 0s 3ms/step - accuracy: 0.7502 - loss: 0.4122 - precision: 0.1405 - recall: 0.8553 - val_accuracy: 0.7408 - val_loss: 0.4450 - val_precision: 0.1577 - val_recall: 0.8085
Epoch 26/30
103/103 ----- 0s 3ms/step - accuracy: 0.7502 - loss: 0.4084 - precision: 0.1436 - recall: 0.8816 - val_accuracy: 0.7311 - val_loss: 0.4624 - val_precision: 0.1581 - val_recall: 0.8511
Epoch 27/30
103/103 ----- 1s 5ms/step - accuracy: 0.7416 - loss: 0.4098 - precision: 0.1387 - recall: 0.8750 - val_accuracy: 0.7335 - val_loss: 0.4511 - val_precision: 0.1566 - val_recall: 0.8298
Epoch 28/30
103/103 ----- 1s 6ms/step - accuracy: 0.7483 - loss: 0.4078 - precision: 0.1442 - recall: 0.8947 - val_accuracy: 0.7311 - val_loss: 0.4609 - val_precision: 0.1581 - val_recall: 0.8511
Epoch 29/30
103/103 ----- 1s 5ms/step - accuracy: 0.7480 - loss: 0.4039 - precision: 0.1433 - recall: 0.8882 - val_accuracy: 0.7408 - val_loss: 0.4397 - val_precision: 0.1577 - val_recall: 0.8085
Epoch 30/30
103/103 ----- 1s 6ms/step - accuracy: 0.7486 - loss: 0.4024 - precision: 0.1429 - recall: 0.8816 - val_accuracy: 0.7531 - val_loss: 0.4188 - val_precision: 0.1586 - val_recall: 0.7660
Epoch 30/30
103/103 ----- 1s 5ms/step - accuracy: 0.7557 - loss: 0.4014 - precision: 0.1472 - recall: 0.8882 - val_accuracy: 0.7408 - val_loss: 0.4390 - val_precision: 0.1577 - val_recall: 0.8085

```

Figure 30: Epoch

After training the neural network we tested it using testing data. The predicted probabilities were converted into class labels using a threshold of 0.5. The final testing accuracy, precision, recall and f1-score were recorded so that the neural network could be compared with logistic regression, naive bayes and KNN.

```
# Test Neural Network model
nn_prob = nn_model.predict(X_test_nn)

nn_pred = (nn_prob > 0.5).astype(int).reshape(-1)

nn_accuracy = accuracy_score(y_test, nn_pred)
nn_precision = precision_score(y_test, nn_pred, zero_division=0)
nn_recall = recall_score(y_test, nn_pred, zero_division=0)
nn_f1 = f1_score(y_test, nn_pred, zero_division=0)

print("Neural Network Results")
print("Accuracy:", nn_accuracy)
print("Precision:", nn_precision)
print("Recall:", nn_recall)
print("F1-score:", nn_f1)
```

... 32/32 ————— 0s 4ms/step
Neural Network Results
Accuracy: 0.7299412915851272
Precision: 0.12582781456953643
Recall: 0.76
F1-score: 0.2159090909090909

Figure 31: Neural Network testing result using class weights

After applying the class weights the neural network achieved an accuracy of 0.7299, precision of 0.1258, recall of 0.76 and f1-score of 0.2159. Compared to previous network result, the accuracy decreased but the recall improved significantly. This means that the model became better at identifying actual stroke cases. Since our dataset was highly imbalanced this trade off is acceptable because accuracy alone can be misleading. In a stroke prediction problem recall is important because missing actual stroke cases can be a serious problem.

2) Logistic regression:

Logistic regression is our second supervised model since it is suitable for binary classification. Our target variables have only two possible classes 0 for no stroke and 1 for stroke. The model will predict the probability of a patient belonging to the stroke and no stroke class. In this project logistic regression model was trained using the scaled training dataset and tested using the scaled testing dataset. As our dataset itself is highly imbalance we used `class_weight="balanced"` so the model gives more importance to the minority stroke class during training. This was done to reduce the chance of the model focusing only on the majority no stroke class. After training the model was tested using testing data. Accuracy, precision, recall and f1 score were calculated to evaluate its performance. These metrics are important because accuracy alone can be misleading for an imbalance healthcare dataset.

```
# Logistic Regression model
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=1,
    class_weight='balanced'
)

# Train the model
logistic_model.fit(X_train_scaled, y_train)

# Predict on test data
logistic_pred = logistic_model.predict(X_test_scaled)

# Predict probabilities for ROC/AUC later
logistic_prob = logistic_model.predict_proba(X_test_scaled)[:, 1]

# Evaluation metrics
logistic_accuracy = accuracy_score(y_test, logistic_pred)
logistic_precision = precision_score(y_test, logistic_pred, zero_division=0)
logistic_recall = recall_score(y_test, logistic_pred, zero_division=0)
logistic_f1 = f1_score(y_test, logistic_pred, zero_division=0)

print("Logistic Regression Results")
print("Accuracy:", logistic_accuracy)
print("Precision:", logistic_precision)
print("Recall:", logistic_recall)
print("F1-score:", logistic_f1)

*** Logistic Regression Results
Accuracy: 0.7377690802348337
Precision: 0.1267123287671233
Recall: 0.74
F1-score: 0.21637426900584794
```

Figure 32: Logistic Regression training and testing result

The logistic regression model achieved an accuracy of 0.7378, precision of 0.1267, recall of 0.74 and f1 score of 0.2164. Since `class_weight="balanced"` was used the model gave more importance to minority stroke class. The recall value shows that the model was able

to identify many actual stroke cases. However the precision is low which means some no stroke patients were predicted as stroke. For this imbalanced healthcare dataset, recall is the especially important because missing actual stroke cases can be a serious issue.

3) K-Nearest-Neighbors (KNN)

We used KNN as our third supervised classification model. It predicts a class of a datapoint by comparing it with nearby data points in the feature space. KNN is good for binary classification and since our target has only two classes 0 for no stroke and 1 for stroke it works well. In our project KNN model was trained using the scaled training dataset and tested using the scaled testing dataset. Scaling was important because KNN uses distance based calculation if the features were not scaled than features that have larger numerical ranges could dominate the distance calculation and affect the prediction. We used “n_neighbors=5” which basically means the model checks the nearest 5 neighbours before deciding the class of a test datapoint. After training the model was tested using the testing data. Accuracy, recall and f1 score were calculated evaluate the performance.

```

# K-Nearest Neighbors model
knn_model = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn_model.fit(X_train_scaled, y_train)

# Predict on test data
knn_pred = knn_model.predict(X_test_scaled)

# Predict probabilities for ROC/AUC later
knn_prob = knn_model.predict_proba(X_test_scaled)[:, 1]

# Evaluation metrics
knn_accuracy = accuracy_score(y_test, knn_pred)
knn_precision = precision_score(y_test, knn_pred, zero_division=0)
knn_recall = recall_score(y_test, knn_pred, zero_division=0)
knn_f1 = f1_score(y_test, knn_pred, zero_division=0)

print("KNN Results")
print("Accuracy:", knn_accuracy)
print("Precision:", knn_precision)
print("Recall:", knn_recall)
print("F1-score:", knn_f1)

... KNN Results
Accuracy: 0.9461839530332681
Precision: 0.0
Recall: 0.0
F1-score: 0.0

```

Figure 33: KNN training and testing result

KNN achieved an accuracy of 0.9462, precision of 0, recall of 0 and f1 score of 0. Although we did get a high accuracy but still precision, recall and f1 score show that KNN model was not able to successfully identify the stroke class. This happened because our dataset is highly imbalanced and most patients belongs in the no stroke class. As a result KNN mostly predicted no stroke. This output shows us once again that high accuracy is not enough.

4) Naive Bayes:

Naive Bayes is our fourth supervised classification model. It uses a probability based classification model that uses Bayes theorem to make predictions. Since our project is a binary classification problem Naive Bayes is great to be used to classify patients into the no stroke or the stroke class. In our project Naive Bayes was trained using scaled training dataset and tested using scaled testing dataset. After training the model predicted the class labels for the testing data. Accuracy, precision, recall and F1-score were calculated to

evaluate the performance of the model since our dataset was highly imbalanced and accuracy alone is not enough to decide how well our model performs.

```
# Naive Bayes model
nb_model = GaussianNB()

# Train the model
nb_model.fit(X_train_scaled, y_train)

# Predict on test data
nb_pred = nb_model.predict(X_test_scaled)

# Predict probabilities for ROC/AUC later
nb_prob = nb_model.predict_proba(X_test_scaled)[:, 1]

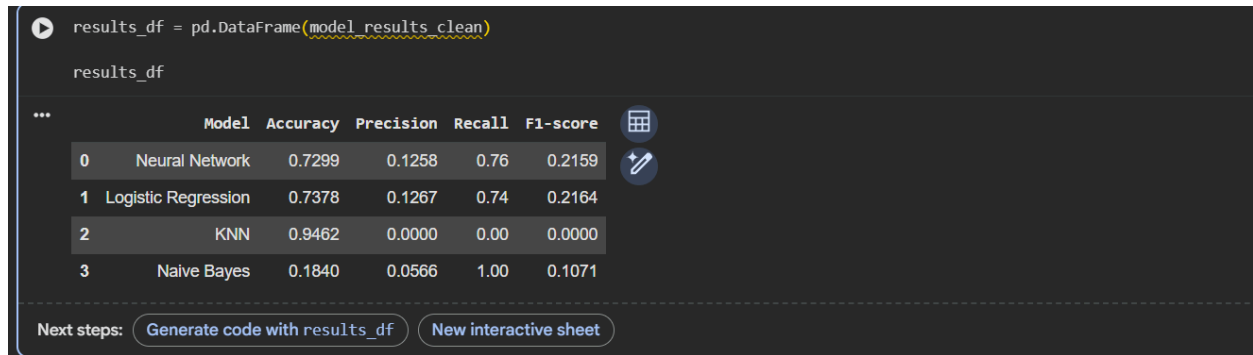
# Evaluation metrics
nb_accuracy = accuracy_score(y_test, nb_pred)
nb_precision = precision_score(y_test, nb_pred, zero_division=0)
nb_recall = recall_score(y_test, nb_pred, zero_division=0)
nb_f1 = f1_score(y_test, nb_pred, zero_division=0)

print("Naive Bayes Results")
print("Accuracy:", nb_accuracy)
print("Precision:", nb_precision)
print("Recall:", nb_recall)
print("F1-score:", nb_f1)
```

```
... Naive Bayes Results
Accuracy: 0.18395303326810175
Precision: 0.05656108597285068
Recall: 1.0
F1-score: 0.10706638115631692
```

Figure 34: Naive Bayes training and testing result

Naive Bayes got a accuracy of 0.1840, precision of 0.0566, recall of 1.0 and F1-score of 0.1071 which clearly shows that recall value is very high which means our model was able to identify all actual stroke cases in the testing data. But the precision is very low which means many no stroke patients were also predicted as stroke and so because of this the overall accuracy is low. This shows that Naive Bayes is sensitive toward the stroke class but produced many false positives predictions.



```
results_df = pd.DataFrame(model_results_clean)
```

results_df

	Model	Accuracy	Precision	Recall	F1-score
0	Neural Network	0.7299	0.1258	0.76	0.2159
1	Logistic Regression	0.7378	0.1267	0.74	0.2164
2	KNN	0.9462	0.0000	0.00	0.0000
3	Naive Bayes	0.1840	0.0566	1.00	0.1071

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

Figure 35: Supervised model result table

After training and testing all supervised models the results can be summarized in a comparison table which has accuracy, precision, recall and F1-score for Neural Network, Logistic Regression, KNN and Naive Bayes. We used class weighted neural network because in the first neural network model result gave a precision, recall and f1-score equal to 0. As our dataset is highly imbalanced, accuracy alone is not enough to select the best model. Recall and F1-score are equally important because this project predicts stroke where missing actual stroke cases can be a serious problem.

Model training and testing (unsupervised):

1) K means clustering

The main task of our project was supervised stroke prediction but the project guideline asked us to treat this problem as an unsupervised learning problem and apply K Means clustering and so we did not use the target column stroke. Instead K means was applied only on the input features. The input data used for K means was the scaled training feature set. The shape of the K means input was (4088, 16) which means 4088 training samples and 16 input features were used for clustering.

```
▶ X_kmeans = X_train_scaled.copy()

print("Shape of K-Means input data:", X_kmeans.shape)

... Shape of K-Means input data: (4088, 16)
```

Figure 36: Input data shape for K-Means clustering

K means clustering was done with `n_clusters=2`. Two clusters were selected because the original supervised target has two main patient outcomes which are no stroke and stroke but the actual stroke label was not used during clustering. K means grouped the patients only based on feature similarity. The cluster count shows that cluster 1 had 2614 patients and cluster 0 has 1474 patients which mean the clustering algorithm divided the patients into two groups of different sizes.

```
▶ kmeans = KMeans(n_clusters=2, random_state=1, n_init=10)

kmeans.fit(X_kmeans)

cluster_labels = kmeans.labels_

print("Cluster labels:")
print(cluster_labels)

print("Cluster counts:")
print(pd.Series(cluster_labels).value_counts())

... Cluster labels:
[0 0 1 ... 0 1 0]
Cluster counts:
1    2614
0    1474
Name: count, dtype: int64
```

Figure 37: K-Means cluster labels and cluster counts

The clustering quality was evaluated using Silhouette score and davies bouldin index.

The silhouette score was 0.2027 which suggests that the clusters do have some separation

but the separation is not very strong. The davies bouldin index was 1.6332 and in davies bouldin index a lower value indicates better clustering quality so the value we got shows that the clusters are usable for analysis but not perfectly separated.

```
silhouette = silhouette_score(X_kmeans, cluster_labels)
dbi = davies_bouldin_score(X_kmeans, cluster_labels)

print("Silhouette Score:", silhouette)
print("Davies-Bouldin Index:", dbi)

... Silhouette Score: 0.20265443573575484
    Davies-Bouldin Index: 1.6332247176927135
```

Figure 38: K-Means clustering evaluation scores

Our dataset has many input features after preprocessing, PCA was used only for visualization. PCA reduced the feature space into two components so that the K means divided the patients into two main clusters. The two clusters are mostly separated along the first PCA component although there is some overlap near the middle.

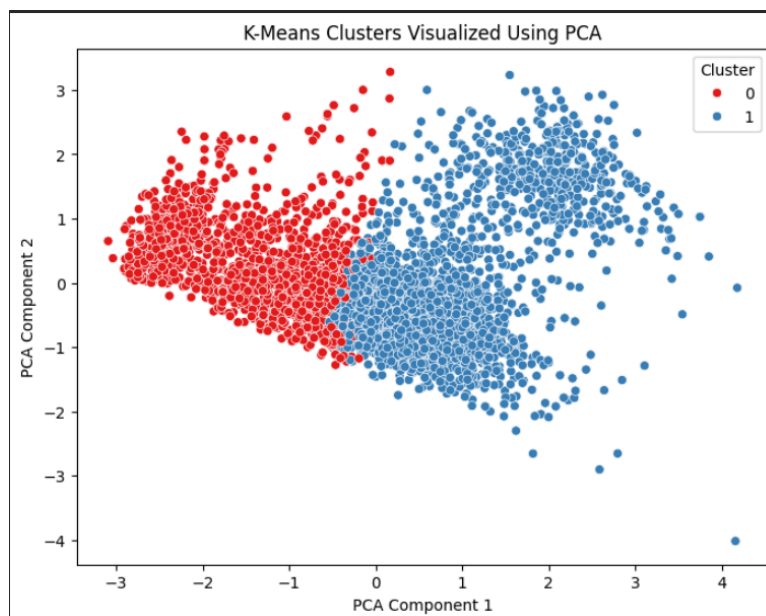


Figure 39: K-Means clusters visualized using PCA

After clustering, the cluster labels were compared with the actual stroke labels only for interpretation. The actual stroke labels were not used while training the K means model. The comparison table shows that cluster 0 contained 1471 no stroke patients and only 3 stroke patients on the other hand cluster 1 had 2418 no stroke patients and 196 stroke patients. This suggests that cluster 1 contains most of the actual stroke cases in the training data so it may represent a relatively higher risk patient group compared to cluster 0. However since K means is an unsupervised method these clusters should not be treated as direct stroke predictions. They only show patient groups based on similarity.

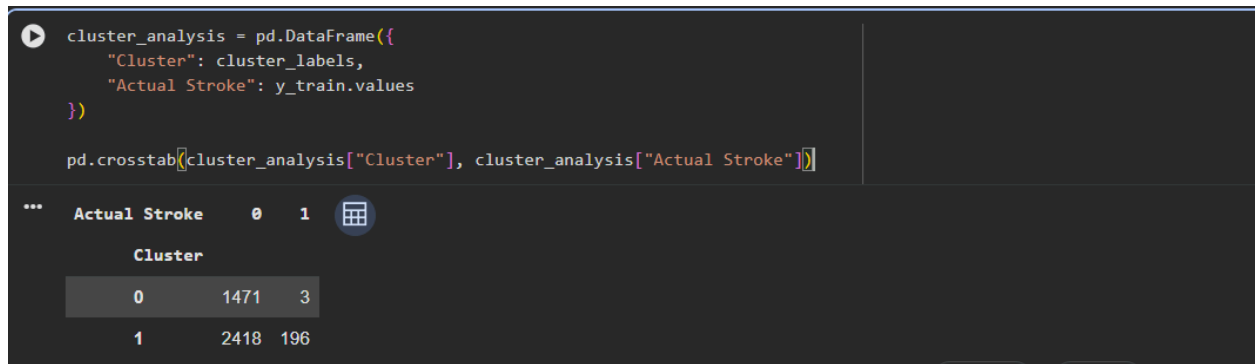


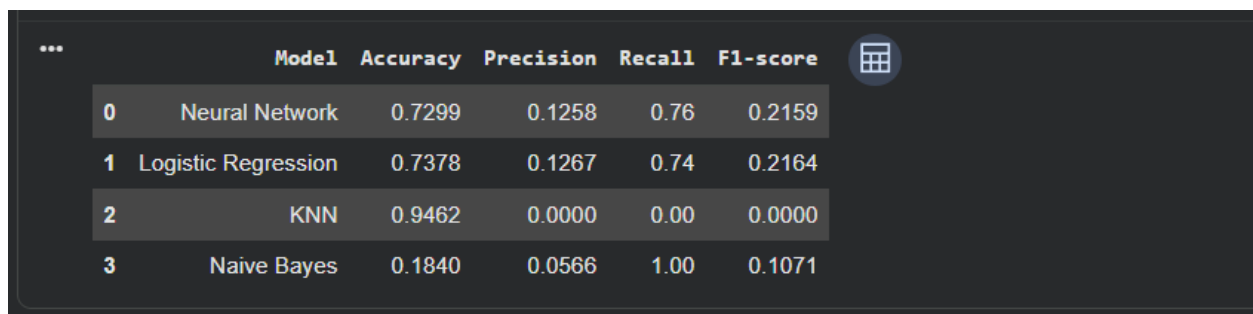
Figure 40: Comparison between K-Means clusters and actual stroke labels

Overall the result of K means shows us that unsupervised learning can separate patients into meaningful groups based on their features. Although K means does not directly predict stroke the cluster comparison suggests that one cluster contains more stroke cases than the other and this supports the idea that clustering can indeed help with patient risk group analysis.

Model selection/ Comparison analysis:

After we trained and tested all the supervised models we compared their performance using different evaluation metrics. Our project was a classification problem and so we used accuracy, precision, recall, f1-score, confusion matrix, ROC curve and AUV score. R^2 score and regression loss were not used because those metrics are used for regression problems. Our project simply predicts whether a patient belongs to the stroke or no stroke class.

Our dataset was highly imbalanced, accuracy alone was not enough to choose the best model. A model can get high accuracy by predicting most patients as no stroke but that does not mean it correctly identified actual stroke cases. So precision, recall, f1-score, confusion matrix, ROC curve and AUC score are also important for model comparisons.

A screenshot of a table comparing the performance of four supervised models: Neural Network, Logistic Regression, KNN, and Naive Bayes. The table has six columns: an index, the model name, Accuracy, Precision, Recall, and F1-score. KNN has the highest accuracy (0.9462) but zero precision, recall, and F1-score. Naive Bayes has the highest recall (1.0) but low accuracy and precision. Neural Network and Logistic Regression have similar performance metrics.

	Model	Accuracy	Precision	Recall	F1-score
0	Neural Network	0.7299	0.1258	0.76	0.2159
1	Logistic Regression	0.7378	0.1267	0.74	0.2164
2	KNN	0.9462	0.0000	0.00	0.0000
3	Naive Bayes	0.1840	0.0566	1.00	0.1071

Figure 41: Supervised model performance comparison table

The performance comparison table above shows that KNN had the highest accuracy with a score of 0.9462 but its precision, recall and f1-score were all 0 which means it failed to identify the stroke class. Neural network and logistic regression on the other hand had lower accuracy compared to KNN but both models had much better recall for the stroke class. Naive Bayes achieve the highest recall of 1.0 but its accuracy and precision were very low.

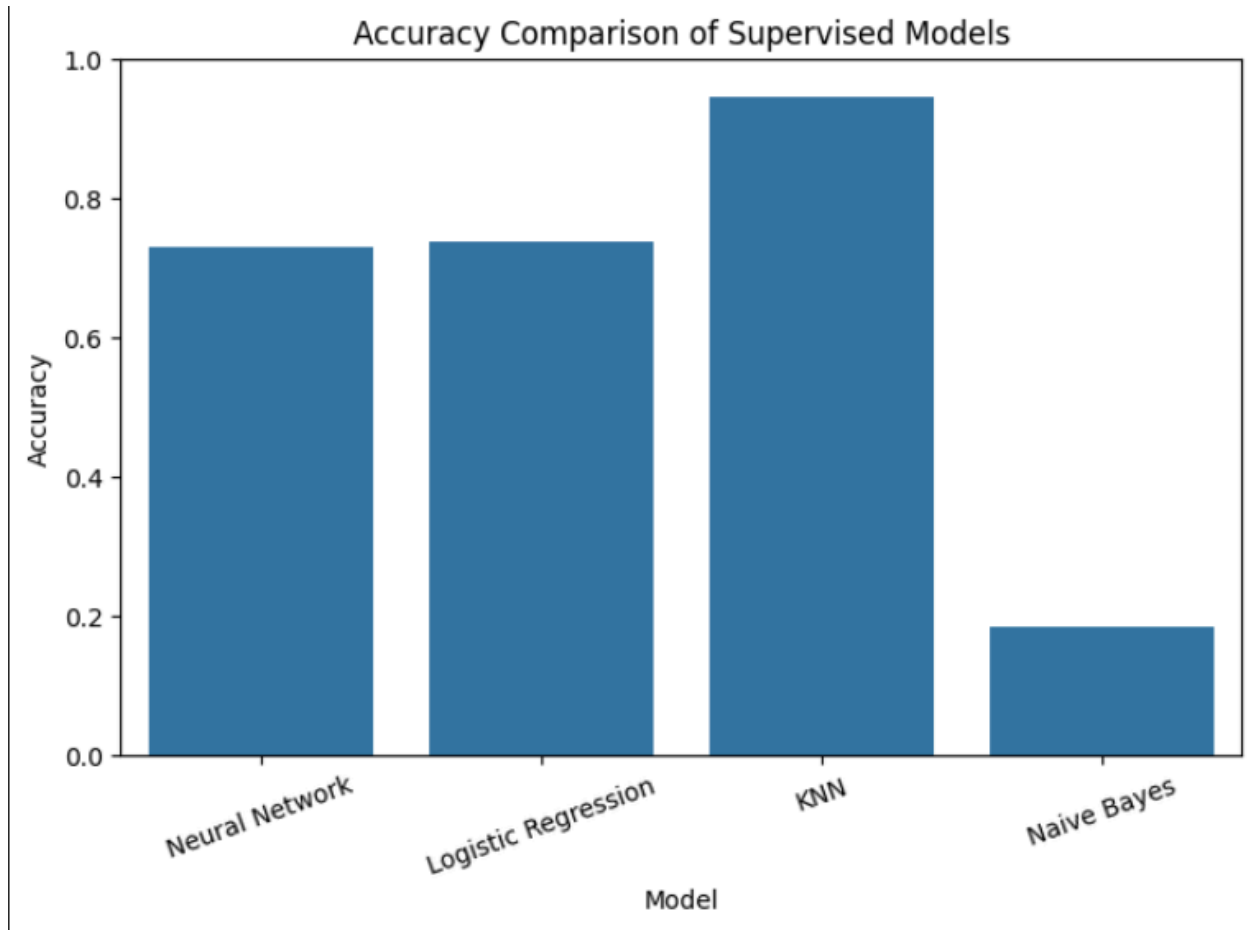


Figure 42: Accuracy comparison of supervised models

From the accuracy comparison above we can see that KNN had the highest accuracy. In second position logistic regression had second highest accuracy and third highest accuracy was of Neural network but Naive Bayes had the lowest accuracy. However this accuracy result is not enough and can be misleading because KNN achieved high accuracy by mostly predicting the majority as no stroke class. Therefore we cannot select the best model based only on accuracy.

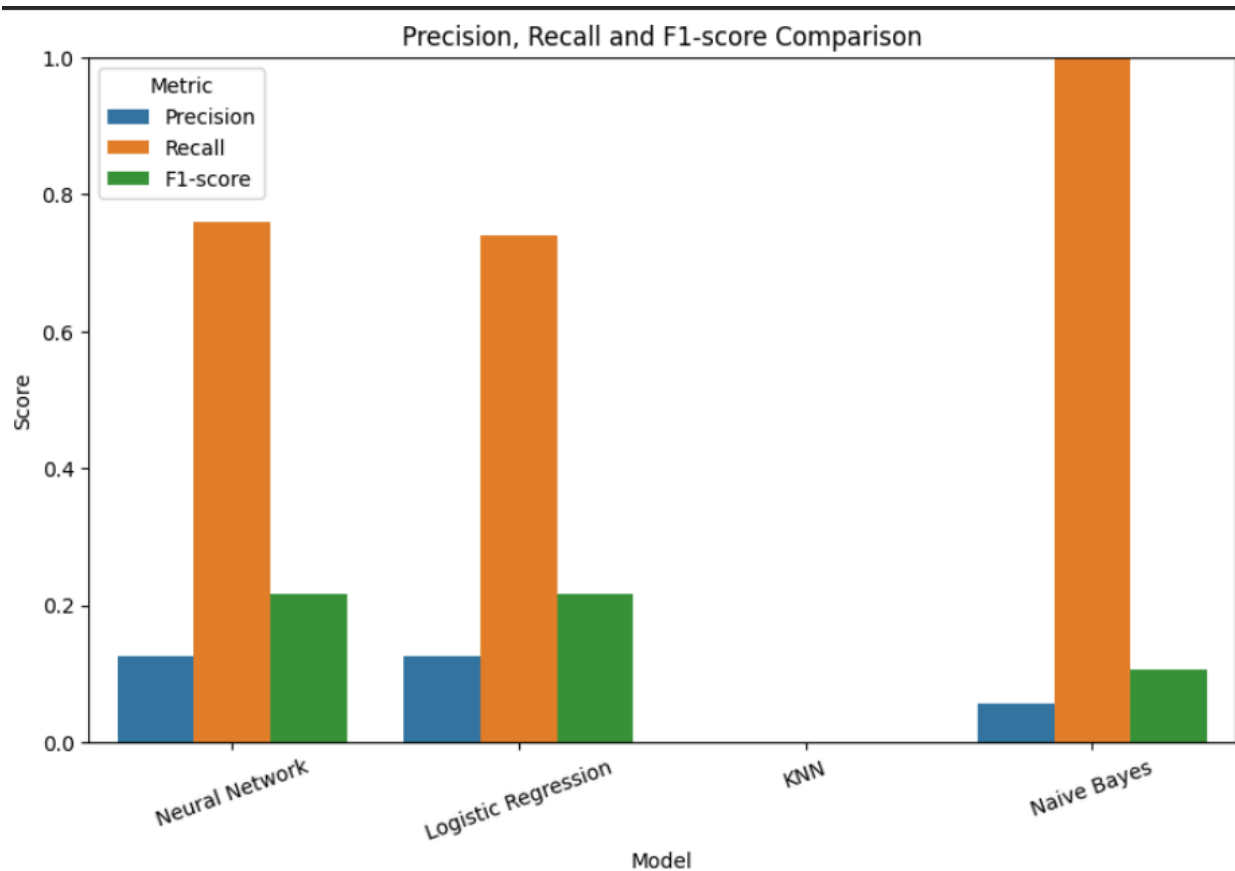


Figure 43: Precision, recall and F1-score comparison of supervised models

Precision, recall and f1 score comparison gives as a better understanding of model performance. KNN has 0 recall, f1 score which means it does not detect any actual stroke cases. Naive Bayes has the highest recall but its precision is very low which means it predicted many no stroke patients as stroke. Neural network and logistic regression show a better balance because both models have relatively high recall and better F1 score compared to Naive bayes and KNN.

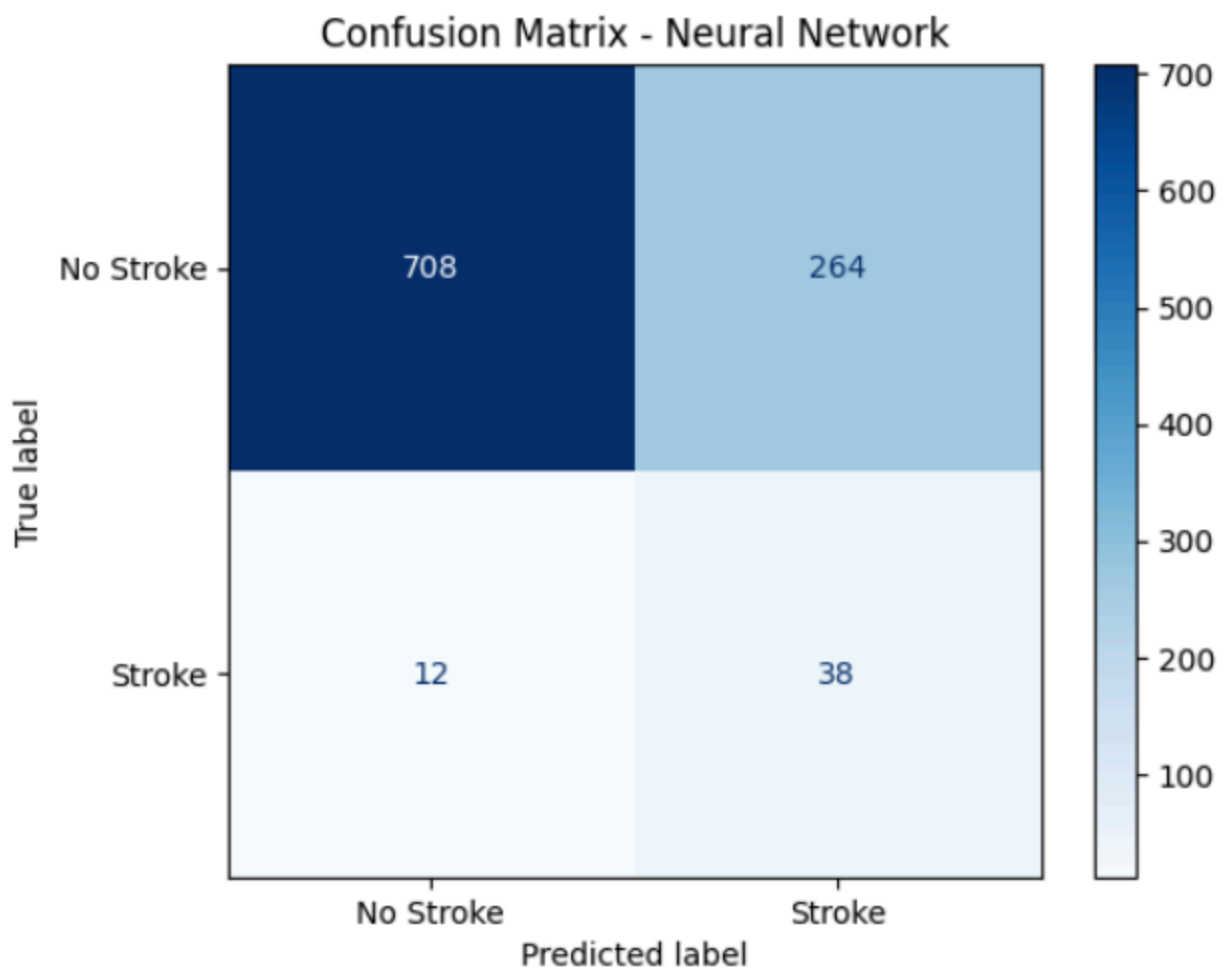


Figure 44: Confusion matrix for Neural Network

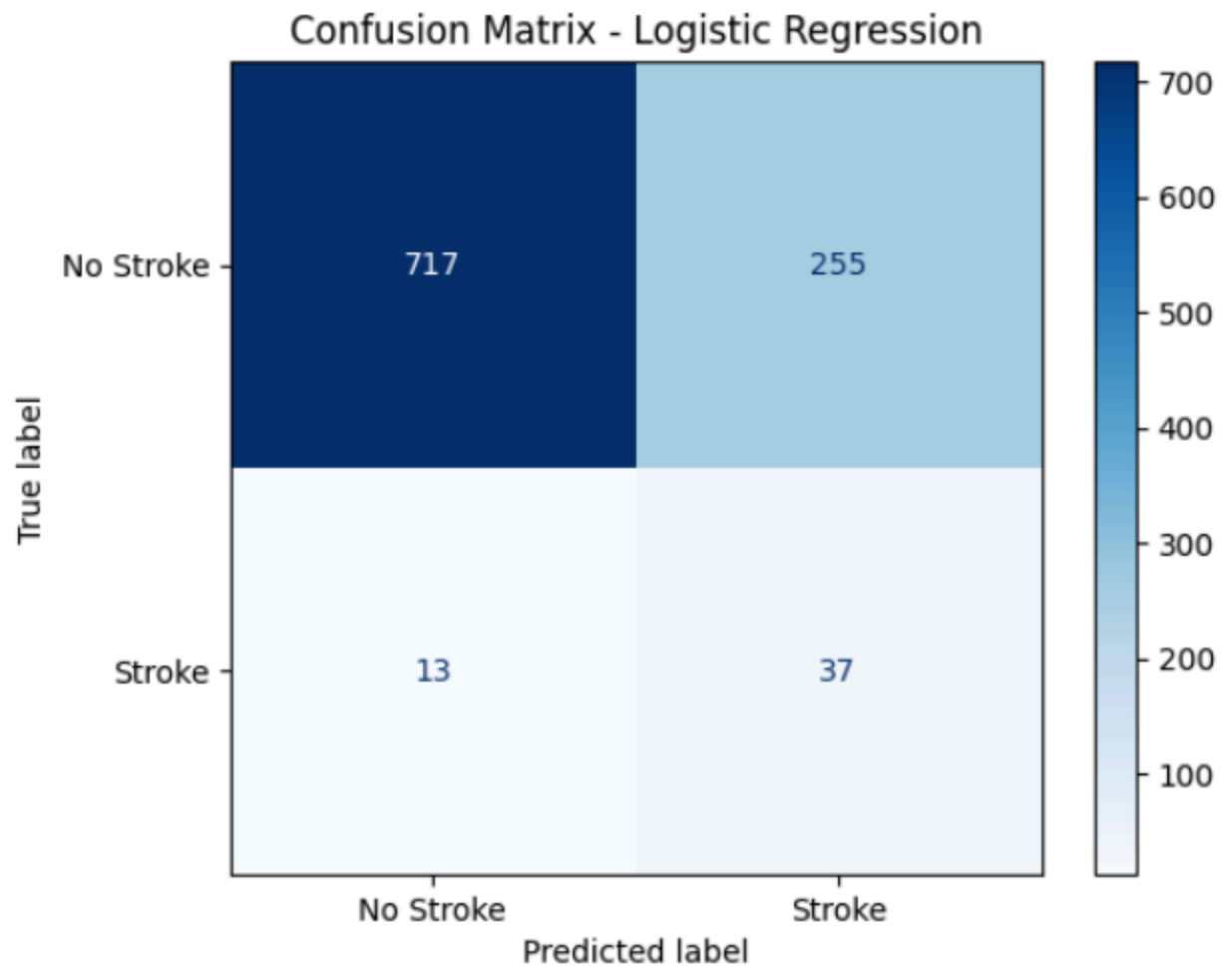


Figure 45: Confusion matrix for Logistic Regression

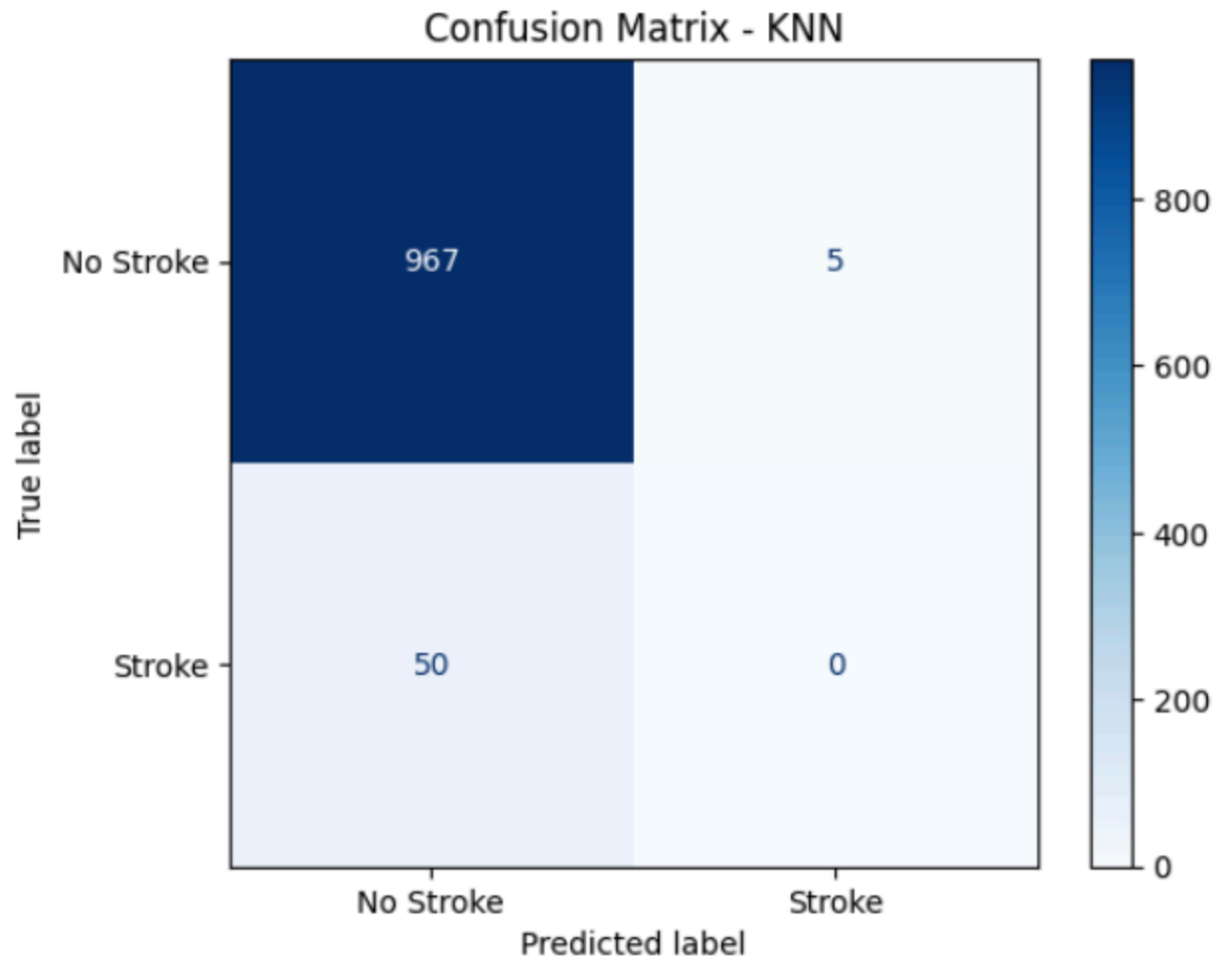


Figure 46: Confusion matrix for KNN

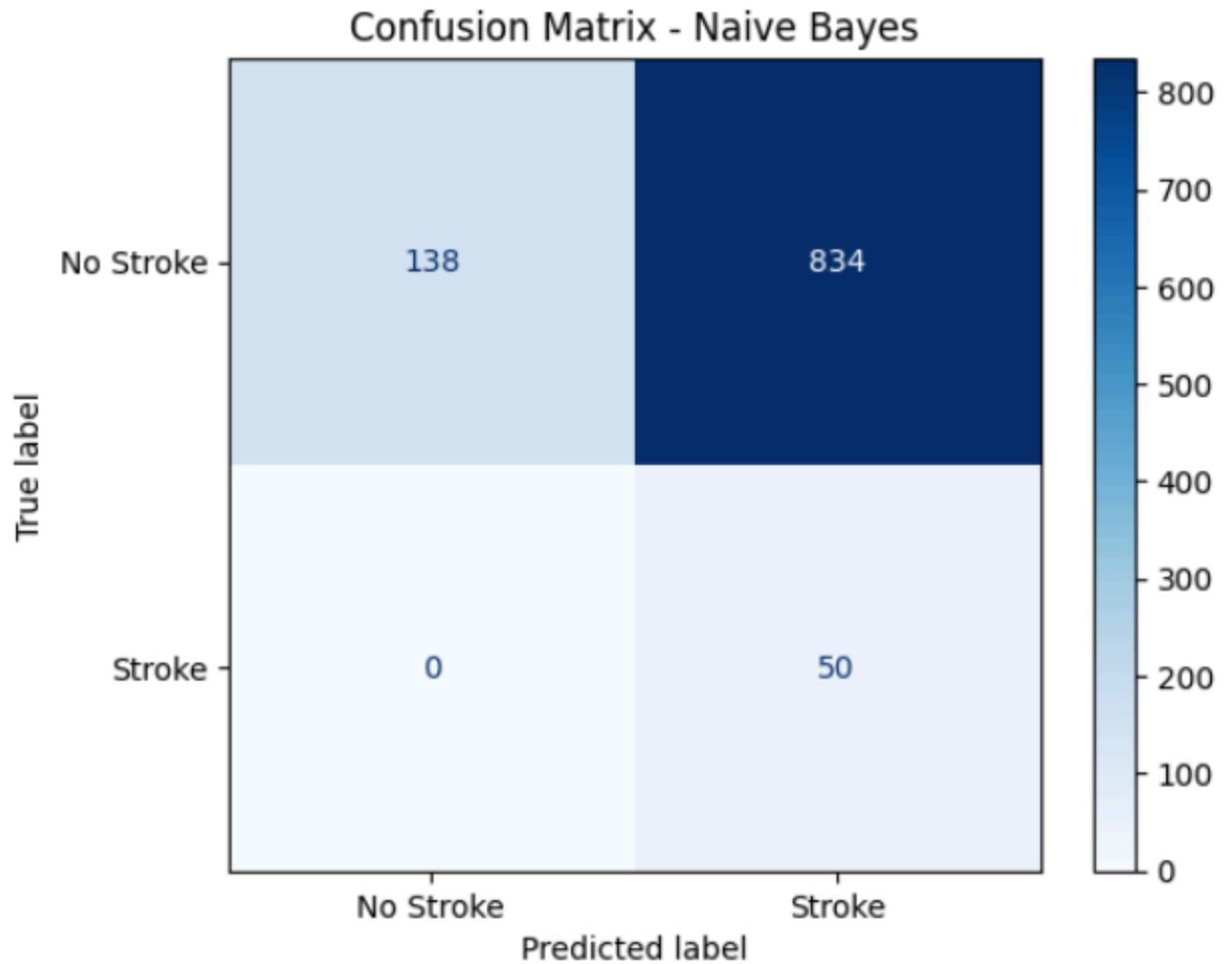


Figure 47: Confusion matrix for Naive Bayes

The confusion matrix shows us how each model has been classified for no stroke and stroke patients. Neural network correctly identified 38 stroke patients and missed 12 stroke patients. Logistic regression correctly identified 37 stroke patients and missed 13 stroke patients. KNN failed to identify an stroke patient because it predicted all 50 actual stroke cases as no stroke. Naive Bayes correctly identified all 50 stroke patients but it all incorrectly predicted that 834 no stroke patients had stroke.

This shows us that KNN is not suitable for this imbalanced dataset because it completely misses the minority stroke class. Naive bayes on the other hand is very sensitive to stroke cases but it

produces too many false positives. Neural network and logistic regression provided a better balance between detecting stroke cases and avoided too many incorrect stroke predictions.

*** Neural Network					
	precision	recall	f1-score	support	
No Stroke	0.98	0.73	0.84	972	
Stroke	0.13	0.76	0.22	50	
accuracy			0.73	1022	
macro avg	0.55	0.74	0.53	1022	
weighted avg	0.94	0.73	0.81	1022	

Logistic Regression					
	precision	recall	f1-score	support	
No Stroke	0.98	0.74	0.84	972	
Stroke	0.13	0.74	0.22	50	
accuracy			0.74	1022	
macro avg	0.55	0.74	0.53	1022	
weighted avg	0.94	0.74	0.81	1022	

KNN					
	precision	recall	f1-score	support	
No Stroke	0.95	0.99	0.97	972	
Stroke	0.00	0.00	0.00	50	
accuracy			0.95	1022	
macro avg	0.48	0.50	0.49	1022	
weighted avg	0.90	0.95	0.92	1022	

Naive Bayes					
	precision	recall	f1-score	support	
No Stroke	1.00	0.14	0.25	972	
Stroke	0.06	1.00	0.11	50	
accuracy			0.18	1022	
macro avg	0.53	0.57	0.18	1022	
weighted avg	0.95	0.18	0.24	1022	

Figure 48: Classification reports of supervised models

The classification report supports the same observation we made earlier. KNN has high overall accuracy but 0 performance for the stroke class. Naive Bayes has a perfect recall with 1 score but low precision. Neural network and logistic regression have similar performance and provide a more balanced result for our project.

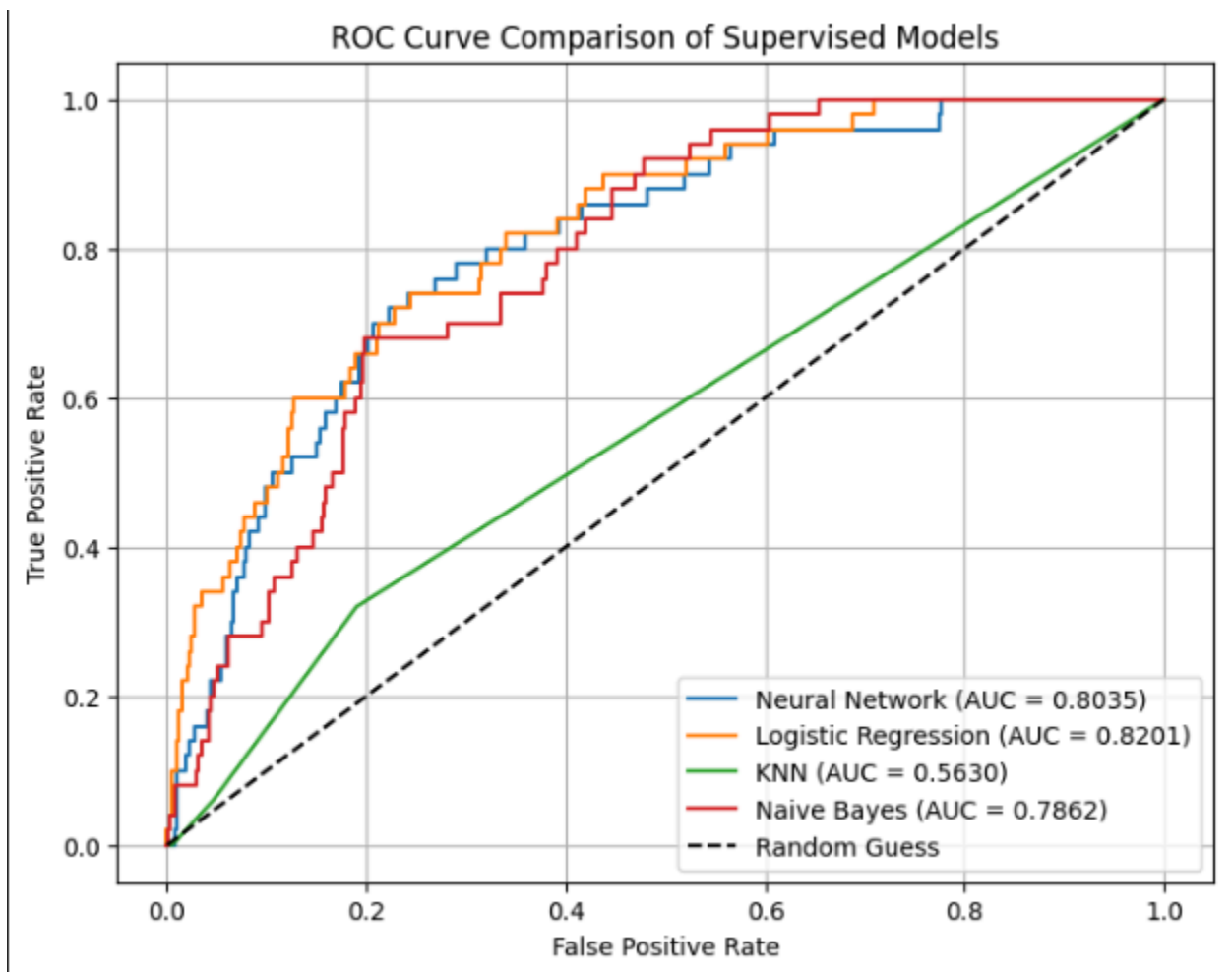


Figure 49: ROC curve comparison of supervised models

The ROC curve compares the ability of each model to separate the stroke and no stroke classes. Logistic regression has the highest AUC score of 0.8201. Second best was by Neural network with 0.8035 score, third best was by Naive bayes with 0.7862 and KNN was in last position with

0.5630. A higher AUC score means that the model has better class separation ability. Therefore Logistic regression performed the best in terms of AUC score.

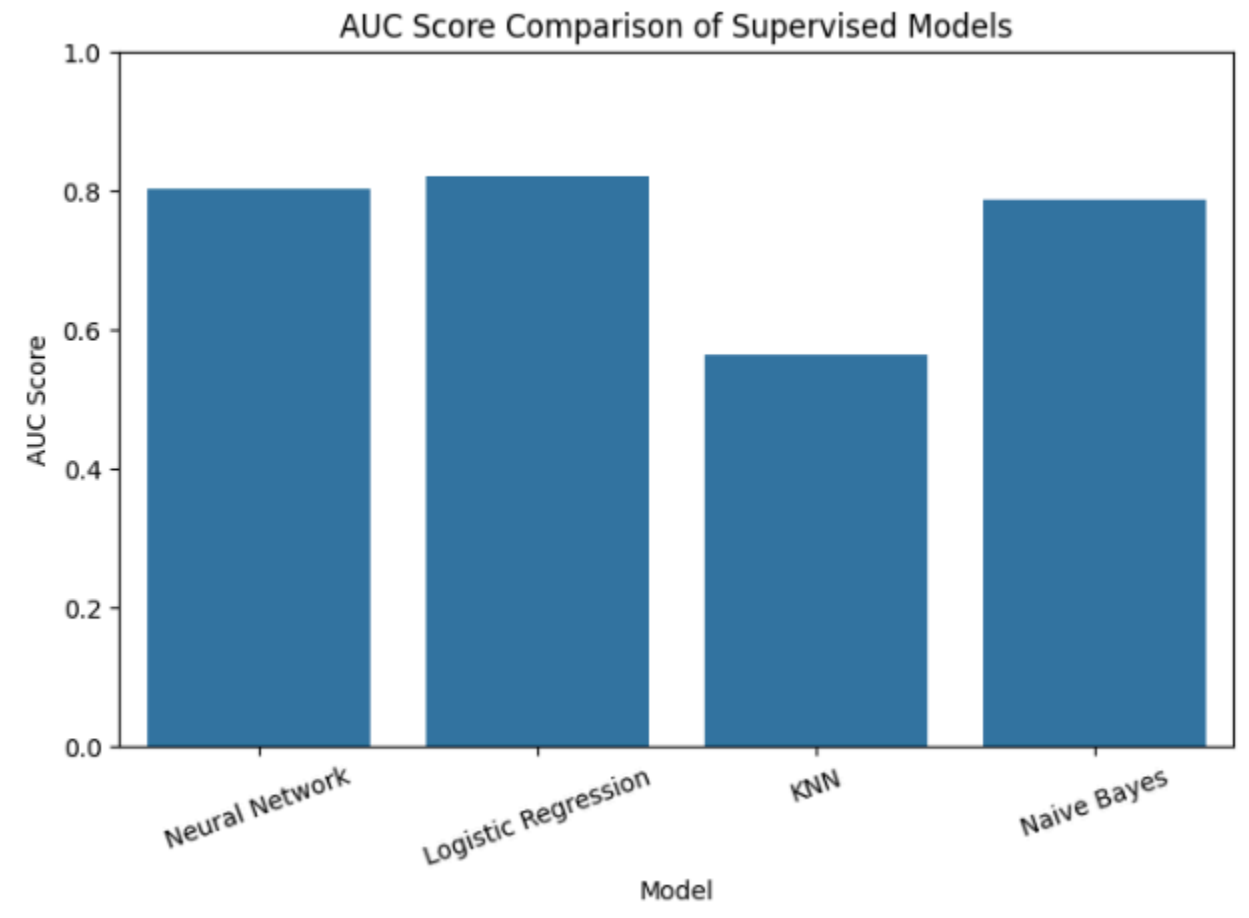


Figure 50: AUC score comparison of supervised models

The area under the curve (AUC score) confirms that logistic regression has the highest AUC value among the other supervised models we used. Neural network also performed well and is close to logistic regression. KNN has the worst AUC score which supports the earlier finding that KNN is not effective for identifying stroke cases in this imbalanced dataset we had.

The area under the curve score table confirms that logistic regression has the highest AUC value among the supervised models. Neural network also performs well and is close to logistic

regression. KNN has the lowest AUC score which supports our earlier finding that KNN is not as effective as the other models for identifying stroke cases in our dataset.

Overall, we found that logistic regression has been our best supervised model for this project because it gives the best AUC score of 0.8201 and the highest f1-score of 0.2164. It also has a recall of 0.74 which means it can identify many actual stroke cases. Neural network also performs well with a recall of 0.76 and AUC of 0.8035 but its f1 score was slightly lower than what we got using logistic regression. Naive Bayes achieved perfect recall but its very low precision and accuracy show it produced too many false positives. KNN had the best accuracy but it failed to detect any stroke cases so it is not a good model for our dataset that is imbalanced.

Hence we can conclude that if we consider accuracy, precision, recall, f1-score, confusion matrix, ROC curve and AUC score then logistic regression is clearly the best overall supervised model for stroke prediction in our project.

Conclusion

From our project we understood that stroke prediction is a challenging binary classification problem especially when the dataset is highly imbalanced. Most patients in the dataset belong to the no stroke class while only a small portion belonged to the stroke class. Due to this imbalance accuracy alone was not enough to judge our models performance. For example KNN achieved the highest accuracy yet it failed to identify actual stroke cases due to its precision, recall and f1 score all being 0 for stroke class. This shows us that a model can look good based on accuracy but still perform poorly for the minority class.

Among the supervised models logistic regression gave us the best overall performance. It achieved the highest AUC score of 0.8201, recall of 0.74 and the highest f1 score of 0.2164. So it meant that logistic regression could identify many actual stroke cases. Neural network also performed well after class weights were applied with a recall of 0.76 and AUC score of 0.8035. Naive Bayes got the best recall score of 1.0 but its precision and accuracy were low which meant that it predicted too many no stroke patients as stroke. Therefore logistic regression was the best overall supervised model for our project.

The results we found were mainly due to severe class imbalance in our dataset. No stroke class had more samples than stroke class so models like KNN mostly focused on the majority class and ultimately failed to detect the minority stroke class. After assigning class weights in neural network and logistic regression the models gave more importance to the stroke class which improved recall but improving recall also caused some false positives predictions which reduced precision. This trade off is expected though because of having an imbalance dataset.

K Means clustering part showed us that unsupervised learning can also provide us with useful insight. K means divided the patients into two clusters based on their feature similarity. One cluster contained more stroke cases than the other which suggested that clustering can actually help identify patient risk groups. However K means should not be treated as a direct stroke prediction model because it does not use the target label during training.

We faced some challenges in our project while handling missing values, encoding categorical features, scaling numerical features correctly and dealing with class imbalance. Another challenge was to choose the best evaluation metric because accuracy is misleading for our dataset. To avoid data leakage by fitting the scaler only on the training data. Overall this project

helped us understand that for healthcare related classification problems model evaluation should focus on recall, precision, f1 score, confusion matrix, ROC curve and AUC score and not only on accuracy.